An independent check of the local Fourier compiler in OpenAI's preprint "An explicit power saving for the exact discrete Fourier transform": exact and numerical tests, depth, cost, and limits

Oleksandr Pylypenko

2026-10-14

Abstract

We independently check one component of OpenAI's preprint "An explicit power saving for the exact discrete Fourier transform", which claims an exact deterministic DFT algorithm at every length n in time $O\big(n(\log n)^{\theta}(\log\log n)^{4-\theta}\big)$, $\theta = 1 - 2.1 \cdot 10^{-13}$, with exact complex arithmetic and unrestricted coefficients. The exponent comes from a tensor network that we did not implement. We implemented the local Fourier compiler (Newton reduction of $F_r(u)$ to triangular Toeplitz factors, the exact-width block Toeplitz word with convolution-DAG replay, and the C-slot shear pattern), whose $O(\log^4 r)$ slot bound gives the factor $(\log\log n)^4$, not θ. All results concern the strict construction, whose topology depends only on the width. Computation in $\mathbb{Q}(\zeta_N)$ proves that the implemented matrix equals $F_r(u)$ for $r = 4, 7, 8, 9, 12, 15, 16, 32$ (the C-slot path for $r = 4, 8, 16$). Exact images in $\mathbb{F}_p$ agree at selected widths up to 512 and, in a streaming test, at $r = 1024$; this is only a necessary condition. Two mpmath precisions 100 digits apart reduce the error by about 100 orders at all widths 7–256. Although $\kappa_2(F_r) = 1$, the word is numerically unstable: $\log_{10}\kappa_2(D_H)$ has leading term cr, $c = \mathrm{Cl}_2(\pi/3)/(\pi\ln 10) \approx 0.1403$, the chain loses 142 digits at $r = 256$, and float64 gives relative error $8 \cdot 10^{23}$ at $r = 64$; the preprint makes no finite-precision claim. Structure hashes coincide for three inputs at 12 widths 64–1024, and a depth model matches the measured ASAP depth exactly at 10 of them. We also recompiled OpenAI's Lean formalisation and audited its statement.

Keywords: discrete Fourier transform; circuit complexity; circuit depth; Toeplitz matrix; numerical stability; cyclotomic fields; exact arithmetic; formal verification (Lean).

Independence statement. This is an independent verification of part of a preprint by OpenAI. As of 2026-10-14 there was no contact with OpenAI about this work; the check was not coordinated with, or reviewed by, the authors of the preprint. No new DFT algorithm is proposed.

1 Introduction

1.1 Motivation

OpenAI's preprint [9] states that the DFT of length n can be computed exactly in time $o(n \log n)$ in a model with unit-cost arithmetic on complex numbers and unit-cost integer operations on words of $O(\log n)$ bits. The Cooley–Tukey algorithm [5] gives $O(n \log n)$. Known lower bounds for related models [1, 2, 8] do not transfer to this model automatically (Section 2.5). The claim is unusual. The preprint itself cautions that the constants of the algorithm are enormous, that the result is asymptotic "with no useful crossover estimate", and that the conclusion does not concern stable floating-point FFTs or bit complexity (`introduction.tex:101`). Recht [11] makes similar practical remarks (hidden constants; exact arithmetic on numbers that grow). It

is therefore worth checking independently at least those parts of the proof that can be checked by computation, and measuring how the limitations of the model that the preprint names itself show up in them.

Source and attribution. The preprint is not a separate article with a DOI. It is a text in the repository `https://github.com/openai/math`, directory `preprints/An-explicit-power-saving-for-the-exact-discrete-Fourier-transform-September-25-2026`: main file `main.pdf`, sources `build/main.tex`, sections `build/sections/*.tex` and bibliography `build/references.bib`. We worked with the state of the directory at commit 301488868 (2026-10-07; repository HEAD fd4aeeb) and refer to statements by their labels (`thm:main`, `cor:decimal`, `loc:compiler`, ...) and to source lines as `file.tex:line`. The author block and the README of the directory give the author as "OpenAI" (`build/main.tex`, `\author{OpenAI}`; `README.md`, "Author: OpenAI"); no individual authors are named. According to the README of the repository, the manuscripts of the collection were produced by an internal OpenAI model. The Lean formalisation that we recompiled (Section 4.8) is a separate artifact of the same repository (directory `lean/`, scope described in `lean/docs/130.md`), not part of the text of the preprint.

1.2 What is checked

We check the local compiler of `local.tex`: the correctness of the implemented operator (exactly and in high-precision arithmetic), its numerical behaviour, its depth under an ASAP schedule, and its cost. Our local experiments do not reproduce the main theorem and do not test the asymptotic saving with exponent θ; separately, we reproduced the mechanical check of the Lean goal (Section 4.8). Table 1 summarises the status of every component of the preprint.

The strength of evidence differs, and we keep the levels apart (Section 4.1). In decreasing order: (1) *proof by computation*, i.e. exact equality in $\mathbb{Q}(\zeta_N)$ for eight widths 4–32 (full chain) and for $r = 4, 8, 16$ (C-slots); (2) *exact but only necessary conditions*, i.e. equality of images in $\mathbb{F}_p$ for selected widths up to 512; (3) *numerical agreement*, i.e. mpmath at two precisions up to $r = 256$; (4) *measurements of the concrete word*: topology, ASAP depth, gates, flops, time. Not checked by local experiments: the network, the asymptotic saving with θ, synchronisation, lengths of all sizes, synthesis. Our results neither confirm nor refute the main theorem.

1.3 Out of scope

We did not implement or check:

- the network of `network.tex` (binary frames, the dimension budget, the tensor power of the kernel C, and `net:tensor-bound`, from which θ comes);
- the synchronisation of tensor factors (`synchronization.tex`);
- lengths of all sizes via the Chinese remainder theorem and chirp convolution (`all-lengths.tex`);
- synthesis and the printer (`synthesis.tex`).

The exponent θ comes from the network, not from the local compiler. The local compiler is a necessary part of the proof (its $O(\log^4 r)$ slots give the factor $(\log\log n)^4$ of the main bound; `synchronization.tex:74`, `all-lengths.tex:137`), but it is not the source of the saving. Our results concern this component and cannot be transferred to the main theorem.

Table 1: Status of the components of the preprint in this study.

Component (preprint file / label)	What is done here	Status
Newton reduction (`loc:newton-reduction`)	Full DFT chain: exactly in $\mathbb{Q}(\zeta_N)$ and $\mathbb{F}_p$; in mp at two precisions	Proof by computation for the whole matrix at $r = 4, 7, 8, 9, 12, 15, 16, 32$ ($\mathbb{Q}(\zeta_N)$); necessary condition at selected $r \leq 128$ ($\mathbb{F}_p$)
Toeplitz word (`loc:toeplitz`), strict mode	Depth, gates, topology (audit), accuracy of Tx	Measured at selected widths up to $r = 1024$; accuracy and scope in Tables 2 to 11
DAG replay (`loc:replay`)	Execution by rounds equals sequential execution; no conflicts	Checked (strict mode)
C-slot pattern (`loc:compiler`)	Own closed-form shear decomposition; zero-test-free splitting; conjugation via DAG with inverse roots	Proof by computation at $r = 4, 8, 16$ ($\mathbb{Q}(\zeta_N)$); necessary condition at selected $r \leq 128$ ($\mathbb{F}_p$); mp up to $r = 256$
Chunk-size rule	Equivalent to the preprint's rule for $b \leq 1024$ (Section 3.3)	Checked for $b \leq 1024$
Concrete network of `network.tex` and its asymptotic saving	Not implemented; numerical θ recomputed (Section 2.2); Lean for a different network in Section 4.8	Not checked by local experiments
Synchronisation (`synchronization.tex`)	Not implemented	Not checked
Lengths of all sizes (`all-lengths.tex`)	Not implemented	Not checked
Synthesis and printer (`synthesis.tex`)	Not implemented	Not checked
Lean formalisation (`lean/OAI/Computability/FourierTransform`)	Compiled: 0 errors, 0 `sorry` warnings; axioms only `propext`, `Classical.choice`, `Quot.sound`	Mechanically checked; comparison of definitions completed within the stated scope; adequacy of `Bill` and the concrete `network.tex` outside the audit

1.4 Contributions

We do not propose a new DFT algorithm. Our contribution is an independent implementation and verification of the local Fourier compiler at selected widths, a quantitative study of its numerical instability, and reproducible tools for exact verification, including streaming evaluation of large computation graphs. Specifically, the study provides:

(a) A reproducible implementation and check of the full chain of the Newton reduction and the Toeplitz word for the DFT: exact in cyclotomic and finite fields, and in high-precision arithmetic, with separate tests of the general Toeplitz operator and of the C-slot path. The evidence levels and checked widths are stated in Section 4.1.

(b) An independently derived algebraic decomposition of a shear through C-slots, with the matrix identity and the nonzero-parameter conditions proved in Section 3.5.

(c) An analytical description of the Fourier-case conditioning of D_H: exact finite partial sums and the Clausen-function asymptotic leading term cr for $\log_{10} \kappa_2(D_H)$ (Appendix B.2).

(d) A quantitative study of digit loss over the stages, growth of intermediate quantities over recursion levels and phases, and sensitivity of the error to phase precision for the checked inputs (Section 4.5). These diagnostics do not provide a complete causal decomposition of rounding error.

(e) Measurements of depth, work, time and memory of the strict construction with verified fixed topology; comparisons with the FFT and direct multiplication in identical cost units; and identification of depth artifacts caused by value-dependent zero-multiplier skipping in our implementation (Section 3.6).

(f) A reproducible streaming exact finite-field checker that evaluates the forward and transposed words without materialising the entire graph, including a separate check at $r = 1024$ on the same two vectors in two fields. Its scope remains an exact necessary-condition test, rather than a proof of the full complex matrix.

These contributions describe the independently developed implementation, analysis and verification tools; no claim of priority is made for the mathematical identities or conditioning asymptotics. The measured limitations (Sections 5.2 and 5.6) concern finite precision or stand-alone execution of the local word, regimes about which the preprint makes no claims. They do not refute the preprint.

2 The claims of the preprint

This section follows the text of the preprint, not our reading of it.

2.1 Model of computation

Following the section "Arithmetic model and main result" of [9]: there are complex registers and integer address registers. One field operation costs one unit. Integer operations and random memory access on words of $O(\log(n+2))$ bits cost one unit. Addition, subtraction and multiplication by prepared scalars that do not depend on the input data are allowed. Division is allowed only in the preparation of scalars, and the denominator must be proved nonzero. One explicitly specified root of unity ζ_D, for $D = D_* < 1024\, n^3$, is supplied at unit cost. There is no restriction on the modulus of coefficients, and the preprint neither considers nor claims stability of floating-point computation: "There is no bound on complex coefficients or intermediate magnitudes, and no finite-precision or numerical-stability assertion" (`introduction.tex:67`). In the same place the preprint cautions that the constants are enormous, that there is no useful crossover estimate, and that the conclusion "does not assert a corresponding improvement for stable floating-point FFTs or for bit complexity" (`introduction.tex:101`).

2.2 Constants and the exponent

The preprint defines

$$m = 10^6, \qquad W_* = 2^{71}, \qquad \Delta = 6871402692000000, \qquad \lambda = m - \frac{\Delta}{W_*}, \qquad \theta = \log_m \lambda, \quad (1)$$

so that $\theta = 0.99999999999978935615699598\ldots < 1$. We write

$$\delta = 1 - \theta \approx 2.106 \cdot 10^{-13}. \quad (2)$$

We recomputed θ independently (mpmath, 50 digits) and obtained the same value.

2.3 Main theorem and decimal corollary

Theorem 1 ([9, Theorem `thm:main`]). *There is a single deterministic exact algorithm which, for every n and every $x \in \mathbb{C}^n$, computes $F_n x$ in time*

$$O\big(n(\log n)^{\theta}(\log\log n)^{4-\theta}\big) \qquad (n \to \infty). \quad (3)$$

The time includes scalar preparation, schedule construction and index work.

Corollary 1 ([9, Corollary `cor:decimal`])**.** *In the same model the transform is computable in time* $O\big(n(\log n)^{1-10^{-13}}\big)$*, which is* $o(n \log n)$*.*

2.4 Role of the local compiler

The main theorem is obtained from the network: `net:tensor-bound` applies $C^{\otimes k}$ to an array of length 2^k in time $O\big(2^k(k+1)^\theta\big)$. The local compiler (`loc:compiler`) and the Toeplitz word (`loc:toeplitz`) do not produce θ, but they are a necessary part of the construction: the words for all axes are padded to $O\big((1+\log r_{\max})^4\big)$ slots, each slot is executed by the network, and this gives the factor $(\log \ell)^4$ in

$$T(n) = O\big(n\,\ell^\theta (\log \ell)^4\big), \qquad \ell = \Theta\Big(\frac{\log n}{\log\log n}\Big) \tag{4}$$

(`synchronization.tex:74--82`; `all-lengths.tex:137`). The gate cost of the local word itself does not enter the bound; only the number of slots and the polynomial preparation do. The local widths are the first ℓ odd primes and one power of two (`all-lengths.tex:13--56`). The preprint states that the exact-width Fourier mechanism is shared with Section 3 of the companion preprint *Finite tensor savings and exact Fourier circuits* [10] (`introduction.tex:158`, `local.tex:10--11`).

What `local.tex` states. For a primitive rth root of unity u put $F_r(u) = (u^{ij})_{0\le i,j<r}$.

- *Lemma* ***`loc:newton-reduction`****.* $F_r(u) = N D_N^{-1} N^{\mathsf T}$ with $D_N = \operatorname{diag}(N_{00}, \dots, N_{r-1,r-1})$, where N is an invertible lower triangular matrix that is a product of two diagonal matrices and one lower triangular Toeplitz matrix. By formula `loc:newton`,

$$N_{ik} = (-1)^k u^{k(k-1)/2}\,\frac{H_i}{H_{i-k}}, \qquad H_j = \prod_{s=1}^{j}(1-u^s), \quad H_0 = 1, \tag{5}$$

  i.e. $N = D_H\, T\, D_c$ with $D_c = \operatorname{diag}(c_k)$, $c_k = (-1)^k u^{k(k-1)/2}$, and $D_N = \operatorname{diag}(c_k H_k)$.

- *Lemma* ***`loc:toeplitz`****.* An invertible lower triangular Toeplitz matrix of width r has a word on exactly r coordinates with $O(1+\log^4 r)$ mixed rounds; "the topology depends only on r". This is an upper bound; the preprint does not claim $\Theta(\log^4 r)$ here.

- *Chunk-size rule* (ibid.). For every $b \in \{1, \dots, v\}$ the intervals are split into chunks of size b (except possibly the last); for every pair the DAG skeleton is printed and its gates g are counted; b is admissible if $g + a + e \le v$ for every pair, where a, e are the target and source chunk sizes; the largest admissible b is chosen. If none is admissible, the direct word is used.

- *Depth recurrence* in the proof:

$$d(v) \le \max\big\{d(\lfloor v/2 \rfloor),\, d(\lceil v/2 \rceil)\big\} + O(1 + \log^3 v). \tag{6}$$

- *Slot pattern* (`loc:compiler`). Every upper shear with nonzero coefficient t has a word with three forward copies of C and diagonal factors (formulas `loc:three-kernel` and `loc:nonzero-shear`): with $H' = \left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$,

$$\begin{aligned} K &= H' \operatorname{diag}(2,1) H' \operatorname{diag}(-3,1) H' = \begin{pmatrix} -8 & -10 \\ 0 & -6 \end{pmatrix}, \\ \begin{pmatrix} 1 & t \\ 0 & 1 \end{pmatrix} &= D_t \operatorname{diag}(-1/8, -1/6)\, K\, D_t^{-1}, \qquad D_t = \operatorname{diag}(4t/5, 1). \end{aligned} \tag{7}$$

An arbitrary coefficient μ is split as

$$\kappa = 1 + \mu\bar{\mu}, \qquad t_1 = \kappa, \qquad t_2 = \mu - \kappa, \tag{8}$$

both nonzero, which gives the same pattern of six C-slots for every coefficient. The conjugate $\bar{\mu}$ is a prepared scalar: the same arithmetic DAG is evaluated a second time with inverse roots.

2.5 Lower bounds cited in the preprint

Morgenstern [8] gives a lower bound for the unnormalised transform with bounded coefficients. Ailon [1, 2] gives bounds for the normalised transform with unitary 2×2 gates and for a model with bounded conditioning. The model of the preprint allows unbounded coefficients, so these bounds do not transfer to it automatically. This explains the absence of a direct contradiction, but by itself does not confirm the proof.

2.6 Related work

Recht [11] (2026-10-12) calls the result "galactic": the hidden constants make it irrelevant at physically attainable sizes, and the model requires multiplying arbitrarily large complex numbers in one operation. The preprint names both limitations itself (`introduction.tex:67`, `:101`), so this critique concerns practical value rather than correctness. Our data (Sections 5.2, 5.6 and 6.1) quantify exactly these limitations for the local component, but do not concern the network and do not establish lower bounds on the cost of any future modification. The displacement-rank construction of chunks is taken by the preprint from Kailath, Kung and Morf [7]. Instability of fast methods for Toeplitz systems has long been known (Bunch [4]); our case is different (multiplication, not solution), but the mechanism is similar: large intermediate quantities. The conditioning of the Newton form depends on the order of the nodes: the natural order $u^0, u^1, \ldots$ of nodes on the circle gives exponential growth, and Leja ordering suppresses it (Reichel [12]). Our formula for $\kappa(D_H)$ (Section 5.2) quantifies the natural order that the preprint uses. We did not investigate whether the node order can be changed without breaking the symmetry of the Newton reduction. Forward and backward errors are used in the sense of Higham [6].

3 Implementation of the local compiler

3.1 Newton reduction

For a primitive root u of order r (in the DFT tests $u = e^{2\pi i/r}$) let H_j be as in (5), $c_k = (-1)^k u^{k(k-1)/2}$ and $T_{ij} = 1/H_{i-j}$ for $i \geq j$. We apply

$$F_r(u) = D_H \, T \, D_{\mathrm{mid}} \, T^{\mathsf{T}} D_H, \qquad D_{\mathrm{mid}} = \mathrm{diag}\big(c_k / H_k\big). \tag{9}$$

This is the identity $F_r(u) = N D_N^{-1} N^{\mathsf{T}}$ of the preprint with $N = D_H T D_c$ and $D_N = \mathrm{diag}(c_k H_k)$, because the central factor is $D_c D_N^{-1} D_c = \mathrm{diag}(c_k^2/(c_k H_k)) = \mathrm{diag}(c_k/H_k)$. The denominators are nonzero because $u^s \neq 1$ for $1 \leq s < r$. Ordinary transposition T^{T} is required here, not the Hermitian adjoint. The word for T^{T} is obtained from the word for T by reversing the order of layers and swapping the ends of every shear (`compiler.transpose_word`). A matrix of width r needs only $h_0, \ldots, h_{r-1}$ with $h_k = 1/H_k$; since $H_r = 0$, h_r is undefined. Our mp code computes h_r but never uses it; in the exact checks we set $h_r = 0$, and the equality holds.

3.2 Block word

A block of width v is split into halves of sizes $\lfloor v/2 \rfloor$ and $\lceil v/2 \rceil$, which are processed recursively on disjoint coordinates. The cross-block part is then processed in chunks of size b: for every

pair (target chunk I from the second half, source chunk J from the first) a convolution DAG of three outer products is built (the displacement construction [7]) and executed by replay with coordinates borrowed from the same block. If no admissible b exists, the block is processed by the direct word.

3.3 Chunk-size rule: ours and the preprint's

Our function `rounds.admissible_chunk` counts gates only for the full $b \times b$ pair and stops at the first b with $g(b) + 2b > v$. The preprint tries all b and checks all pairs, including incomplete last chunks. On the checked range both rules give the same b, for two reasons.

- Every actual pair, including incomplete ones, passes the check $g + a + e \le v$: this is an assertion in `block_word_local` ("pair does not fit its block"), and it never fired in any measurement of the adaptive rule.
- The function $b \mapsto g(b) + 2b$ is nondecreasing for $1 \le b \le 1024$ (checked on the complete tables of $g(b)$ in both modes). Hence there is no admissible b larger than the first inadmissible one minus one. For $b > \lfloor v/2 \rfloor$ there are no full pairs, and the pair of whole halves does not fit because $g > 0$.

Beyond $b \le 1024$ we did not prove monotonicity.

3.4 Counting laws

Our measurements give the following laws; $g(b)$ is the gate count and $L(b)$ the number of replay layers of one $b \times b$ pair.

- Strict mode: $g(1) = 8$; zero-skipping mode: $g(1) = 6$. The strict table of $g(b)$ differs from the zero-skipping one only at $b = 1$; hence, in both modes,

$$g(b) = b\,(24 \log_2 b + 38) \qquad \text{for powers of two } 2 \le b \le 1024. \tag{10}$$

- $L(1) = 28$ in strict mode (18 with zero skipping), $L(2) = 94$, and

$$L(b) = 62 + 32 \log_2 b \qquad \text{for powers of two } 4 \le b \le 128 \tag{11}$$

  (checked for $b = 4, 8, \ldots, 128$ in both modes; in the zero-skipping mode also for $b = 256$ and 1024).

- Consequence of $g(1) = 8$: in strict mode a chunk of size 1 fits only for $v \ge 10$ (not $v \ge 8$). The "chunk 1" rule in the strict measurements uses exactly this condition.

3.5 Shear decomposition through C-slots

We use our own closed-form decomposition of the lower shear. Let $C = \left(\begin{smallmatrix} p & q \\ q & p \end{smallmatrix}\right)$ with $p = (1+i)/2$, $q = (1-i)/2$, and

$$D_0 = \operatorname{diag}\Big(-\frac{5i}{3c},\, 1\Big), \quad D_1 = \operatorname{diag}(2, 1), \quad D_2 = \operatorname{diag}(3, -1), \quad D_3 = \operatorname{diag}\Big(\frac{2c}{5(1+i)},\, \frac{1}{2(1-i)}\Big). \tag{12}$$

Lemma 1. *For every $c \neq 0$,*

$$\begin{pmatrix} 1 & 0 \\ c & 1 \end{pmatrix} = D_0\, C\, D_1\, C\, D_2\, C\, D_3. \tag{13}$$

Proof. Direct multiplication gives

$$C\,D_1\,C\,D_2\,C = \begin{pmatrix} \frac{3}{2}(-1+i) & 0 \\ \frac{5}{2}(1+i) & 2(1-i) \end{pmatrix}. \tag{14}$$

Multiplying by D_0 on the left and D_3 on the right: the $(1,1)$ entry becomes $-\frac{5i}{3c}\cdot\frac{3}{2}(-1+i)\cdot\frac{2c}{5(1+i)} = \frac{-i(-1+i)}{1+i} = 1$, the $(2,2)$ entry becomes $2(1-i)\cdot\frac{1}{2(1-i)} = 1$, and the $(2,1)$ entry becomes $\frac{5}{2}(1+i)\cdot\frac{2c}{5(1+i)} = c$. □

This is an algebraic identity with three copies of C. The preprint uses a different decomposition of the same form (an upper shear through H' and K, (7), also with three copies of C).

For an arbitrary coefficient we split, as the preprint does, $\mu = t_1 + t_2$ according to (8).

Lemma 2. *For every $\mu \in \mathbb{C}$, with $\kappa = 1 + |\mu|^2$, one has $t_1 = \kappa \geq 1$ and*

$$|t_2| = |\mu - \kappa| \;\geq\; \kappa - |\mu| = \left(|\mu| - \tfrac{1}{2}\right)^2 + \tfrac{3}{4} > 0. \tag{15}$$

The proof is in Appendix B.1. This gives six C-slots for every shear without any test for zero.

Conjugation. As in the preprint, the conjugate $\bar{\mu}$ is prepared by building the whole word a second time with inverse roots ($u \mapsto u^{-1}$, every root of order L replaced by its inverse; `mp_exact.ROOT_SIGN = -1`). Because the rational constants are real, the coefficients of the second word equal $\bar{\mu}$. We checked that the structure of the second word coincides with the first and that the relative difference from conj(μ) does not exceed 10^{-59} ($r = 16$, 60 digits); in the exact fields it is zero. In $\mathbb{Q}(\zeta_N)$ conjugation is the automorphism $\zeta \mapsto \zeta^{-1}$. A prime field $\mathbb{F}_p$ has no nontrivial automorphisms; replacing roots by their inverses there simply reduces the same complex-conjugate expression (the image of $\zeta \mapsto \zeta^{-1}$ of the cyclotomic field). The C-slot rows of Table 2 and all of Table 4 use exactly this variant.

3.6 Strict construction and static branch audit

Strict construction. All results in this paper use the strict construction, in which no gate is dropped because of a zero coefficient (`conv_dag.KEEP_ZERO_MULTIPLIERS = True`). This is what the preprint prescribes: the convolution topology "can be printed before any scalar coefficients are evaluated, retaining even gates with a coefficient that might happen to vanish" (`local.tex:140--142`); all three outer products are kept "even when some coefficients vanish" (`local.tex:268--269`); and "every structural choice uses integers alone" (`local.tex:42`).

A variant that skips a pointwise multiplier whenever its coefficient is exactly zero (the condition `abs(coef) == 0` in `conv_dag.py`) does not change the operator, since the skipped multiplier is zero. But it is value-dependent: it needs a zero test on a complex coefficient, which the construction of the preprint does not use, and it makes the topology depend on the coefficients. At $r = 64$ such a variant skipped 2024 of 12 066 pointwise multipliers for one input and 2002 for another. Our earlier implementation did exactly this, and it produced spurious effects: a crossover of the depths of the adaptive and chunk-1 rules between $r = 320$ and 336, and a seed-to-seed spread of the depth of order 10^{-4}. Neither exists in the strict construction (Sections 5.4 and 5.5). The zero-skipping variant appears below only as an explicitly labelled comparison. No correction of our implementation ever revealed an error in the preprint.

Static audit. We reviewed all conditional statements in the modules that build the word (`conv_dag.py`, `chunk_dag.py`, `replay.py`, `rounds.py`, `toeplitz_disp.py`, `mp_exact.py`, `mp_compiler.py`, `generic_word.py`).

- There is exactly one branch on the value of a coefficient: skipping a pointwise multiplier whose coefficient is exactly zero (`conv_dag.py:91`, `mp_exact.py:91`). It is disabled in strict mode. In `generic_word.py` (exact fields) it does not exist at all.
- All other conditions test structure only: whether a node exists (`None` for zero padding to a power of two), the reference type (`'x'` or `'z'`), indices, block sizes, and the result of the chunk rule, which counts gates structurally.
- The thresholds $|c| < 10^{-12}$ (`compiler.py`, float64) and $|c| < 10^{-30}$ (`mp_compiler.mp_compile_blocks`) occur only in functions that are not used for any number in this paper.
- The random DAG generator in `replay.py` (lines 215–231) is used only in tests.

4 Verification methodology

4.1 Four evidence levels

We distinguish four kinds of evidence, which must not be mixed.

(E1) Proof by computation. Exact evaluation in $\mathbb{Q}(\zeta_N)$ with rational coefficients, where equality is equality of complex numbers. Evaluating the word on all basis vectors proves the matrix identity for that r.

(E2) Exact but necessary condition. Exact evaluation in $\mathbb{F}_p$. Since $\mathbb{F}_p$ is the image of a ring homomorphism (Proposition 1), agreement in $\mathbb{F}_p$ is necessary for, but does not prove, the complex identity.

(E3) Numerical agreement. High-precision (mpmath) and float64 arithmetic. Agreement at two precisions is consistent with exact equality but is not a proof.

(E4) Measurement of the concrete word. Topology, ASAP depth, gate counts, flops, time and memory.

Two further kinds of check support these. *Mechanical checks*: the recompilation of the Lean formalisation (Section 4.8) and unit tests of our own code; the current full test discovery has 123 passing tests, including regressions for exact roots, strict mode, phase rounding and the table validator. Unit tests are not by themselves a proof of any theorem of the preprint. *Analytical comparison* of the implementation with the text of the preprint: Sections 2.4 and 3 compare the main notation and list all known deviations; there is no complete line-by-line comparison.

4.2 Exact arithmetic

For checks without rounding we implemented two fields with a common interface (`allength/exact_field.py`) and a word builder over an arbitrary field (`allength/generic_word.py`: the same algorithm as the mp version, but without any skipping of multipliers).

Cyclotomic fields. Elements of $\mathbb{Q}(\zeta_N)$ are polynomials with rational coefficients modulo the Nth cyclotomic polynomial Φ_N; inversion uses the extended Euclidean algorithm with the check $a \cdot a^{-1} = 1$. We take

$$N = \operatorname{lcm}\bigl(r,\, 2^{\lceil \log_2 8r \rceil}\bigr), \tag{16}$$

so that $u = \zeta_r$ and all power-of-two roots of order $< 8r$ required by the convolution DAGs lie in the field. Equality here is equality of complex numbers under the embedding $\zeta_N \mapsto e^{2\pi i/N}$. Checking all basis vectors is a proof by computation that the matrix of the exact builder `generic_word.py` equals $F_r(u)$ for that r.

Finite fields. We take p among the smallest primes of the form $kN + 1$ above 2^{61} and $\omega \in \mathbb{F}_p$ of order N. The reduction $\zeta_N \mapsto \omega$ is a ring homomorphism of $\mathbb{Z}[\zeta_N]$ (not of all of $\mathbb{Q}(\zeta_N)$); it extends to those rational elements whose denominators have nonzero image modulo p. The code inverts and checks $a \cdot a^{-1} = 1$, so a division by the image of zero is detected. For the coefficients of our word these denominators include the H_k, the constants of the C-decomposition and the parameters t_1, t_2 of split shears; during exact execution every inversion is checked to be nonzero modulo the chosen p. The complex argument for $t_1, t_2 \neq 0$ (Lemma 2) does not carry over to $\mathbb{F}_p$, because it uses positivity: the image of $\mu\bar{\mu}$ is not a nonnegative real number there.

Proposition 1. *Let $\mathfrak{p}$ be the kernel of $\mathbb{Z}[\zeta_N] \to \mathbb{F}_p$, $\zeta_N \mapsto \omega$. If a straight-line program over $\mathbb{Q}(\zeta_N)$ with inputs in $\mathbb{Z}[\zeta_N]$ and rational constants with denominators prime to p is executed in $\mathbb{F}_p$ and every inversion there is of a nonzero element, then the $\mathbb{F}_p$ result is the image of the $\mathbb{Q}(\zeta_N)$ result. In particular, every identity that holds over $\mathbb{Q}(\zeta_N)$ also holds in $\mathbb{F}_p$.*

The proof (localisation at $\mathfrak{p}$) is in Appendix B.3. Consequently agreement in $\mathbb{F}_p$ is an exact but only necessary condition; the number of distinct primes is given in Table 2 and in the description of the separate streaming test (Section 5.1).

Negative control. If one coefficient in the middle of the word at $r = 16$ is changed, the exact check in $\mathbb{F}_p$ finds 60 mismatches among the 256 entries of the matrix.

4.3 High-precision arithmetic

We use mpmath with 60–700 digits. Every width of the full DFT was checked at two precisions that differ by 100 digits (Table 3). If the implemented operator equals $F_r(u)$, the error must decrease by a factor of 10^{100}. The relative error is

$$\eta = \frac{\|y - F_r x\|_2}{\|F_r x\|_2}, \tag{17}$$

where $F_r x$ is computed by the direct sum with powers of u. For the DFT the normwise forward and backward errors coincide, because $\|F_r x\|_2 = \sqrt{r}\,\|x\|_2 = \|F_r\|_2 \|x\|_2$; we therefore report only one. "Digits lost" is the number of working digits minus $-\log_{10}\eta$.

4.4 Operator coefficients versus data vectors

We distinguish two things.

- *Operator coefficients h.* For the DFT they are fixed: $h_k = 1/H_k$ for the given r and u. Random h test the general Toeplitz compiler but do not reproduce the structure of the Fourier case; tests of one do not confirm the other.
- *Data vectors.* In $\mathbb{Q}(\zeta_N)$ and $\mathbb{F}_p$ the basis and sampled random vectors are listed in Table 2. In mp all basis vectors were used at $r = 7, 8, 9, 15, 16, 31, 32, 33, 63, 64, 128$, and three random vectors at $r = 96, 127, 256$ (Table 3). Real and imaginary parts of random vectors are uniform on $[-1, 1]$ in mp; in $\mathbb{F}_p$ the entries are uniform over the field.

Toeplitz word with random h (Table 8): real and imaginary parts of h_k and x_k uniform on $[-1, 1]$, h_0 replaced by $|h_0| + 1$, seed $= r$. Strict depth measurements (Tables 10, 11 and 18): h_0 replaced by $|h_0| + 1$, the other $h_k = \mathcal{N}(0, 1) + i\,\mathcal{N}(0, 1)$, seed $= r$; the structure was additionally checked on two more inputs ($h =$ ones and another seed).

4.5 Stage-wise error, growth and phase precision

The chain (9) has five stages: D_H, T^{T}, D_{mid}, T, D_H. A reference run with 150 extra digits gives the exact outputs s_k of every stage. For every stage we measure: the *local loss* (the stage applied at working precision to the exact input s_{k-1}), the *accumulated loss*, the *growth* of intermediate values ($\log_{10}$ of the maximum modulus of a coordinate inside the stage divided by the maximum modulus of its input), and the norm of the output. The conditioning of T and D_H is computed by SVD in mpmath.

Growth decomposition. We build the word with a tag (block width v, phase) on every layer (`causal_levels.py`); the phase is a leaf ($v = 1$), the direct word (block without admissible chunk), or a cross-block chunk (replay of a displacement DAG built from the reciprocal series g). Replaying the word on a reference vector, we attribute to every layer the positive increment of $\log_{10}$ of the running maximum modulus of a coordinate. These increments telescope to $\log_{10}\bigl(\text{global maximum}/\|\text{input}\|_\infty\bigr)$, so the decomposition is additive in the *growth* (not directly in the error) over levels and phases.

Phase-precision control. In `phase_precision.py` the coefficients and the input are prepared at the reference precision DPS + 150 and fixed across runs. For T^{T} the input is $D_H x$; for T it is the reference $D_{\text{mid}} T^{\mathsf{T}} D_H x$, i.e. the actual input of that stage in the DFT chain. Operations of the selected phase are executed at reference precision, the rest at DPS. In a shear the product of coefficient and coordinate and the subsequent addition are rounded separately, as in ordinary execution of the word; this is not a fused multiply–add. The comparison is with execution of the whole word at reference precision. The control measures the influence of the execution precision of phases; it does not measure the error of coefficient preparation, and the reference remains a finite high-precision approximation. For $r = 128$, raising the margin from 150 to 200 digits did not change the rounded results. Both conditions above (separate rounding; the actual stage input) are easy to violate; an earlier version of our control script violated both, and all numbers reported here come from the corrected script.

4.6 Depth as an analogue of rounds

The ASAP schedule places every layer in the earliest round after all previous layers that touch its coordinates. This is the measured depth of this particular word and an upper bound on what a schedule of this word can achieve. It is a measurable analogue of the "mixed rounds" of `loc:toeplitz`, not an identical notion, and we did not prove optimality of the schedule. Large widths are computed in streaming mode (`strict_tables.py`) without storing the word in memory.

4.7 Cost units

A gate is one shear $x_{\text{dst}} \mathrel{+}= c\, x_{\text{src}}$ or one scaling $x_d \mathrel{*}= c$. In real flops, with ordinary complex multiplication, a scaling costs 6 operations and a shear 8. Special coefficients may cost less; we do not account for that. References in the same units:

- radix-2 FFT: $5r \log_2 r$;

- Toeplitz multiplication via circulant embedding of length $2r$: two FFTs of length $2r$ and $2r$ pointwise products, i.e. $2 \cdot 5 \cdot 2r \log_2(2r) + 12r$ (the coefficient spectrum prepared in advance);
- direct triangular Toeplitz multiplication: $r(r+1)/2$ complex multiply–adds, i.e. $4r(r+1)$.

The DFT via the word costs two Toeplitz words plus three diagonal factors ($18r$ flops). The C-slot form costs even more: every shear becomes 8 diagonal factors and 6 C-slots. These numbers are not running times of the preprint's algorithm and do not assess it. In the preprint the local word is never executed alone on one vector: its slots are combined tensorially over all axes and executed by the network, so the bound counts slots, not gates (`synchronization.tex:74--82`); moreover, the preprint's time includes scalar preparation and index work.

4.8 Lean recompilation and statement audit

Build. We built a minimal project: Lean 4.34.1, Mathlib at the fixed commit `d13f23b723b8a846827a245b89c10fc7d3f11612`, and copies of the folders `FourierTransform` and `FourierCircuit` from OpenAI's repository. The import closure of `OAI.Computability.FourierTransform.Main` has 89 modules: 88 from `FourierTransform` and `FourierCircuit/Core.lean`; the external dependency is Mathlib. The original full build executed 9012 tasks without errors or `sorry` warnings; this count also includes modules that `Main` does not import. All 89 modules of its closure were rechecked separately. `#print axioms` for `transform_main`, `convolution_main` and both `_order` variants shows only `propext`, `Classical.choice` and `Quot.sound`.

What is proved. `transform_main` has type `DFTGoal` (`Goal.lean`): there exist fixed programs `order` and `solve` and a function W, independent of the data vector, such that `DFTProgram` and `TimeBounds W` hold. For every $n > 0$, `order` produces from the length n a root order D with $0 < D < 1024\, n^3$; `solve` on input (n, ζ_D, x) returns exactly `dft n x` for every $x \in \mathbb{C}^n$. The condition `d.work`$+1+$`result.work` $\le W(n)$ includes the preparation of the order, one operation supplying the root, and the transform. The polynomial bound $(n+2)^{\texttt{cBound}}$ bounds the integer peak of both programs, D, and $W(n) + 10(n+2)$; it is not a separate explicit bound on array sizes. `TimeBounds W` requires $W = O(\texttt{paperTime})$, $W = O(\texttt{decimalTime})$ and $W = o(n \log n)$, where $\texttt{paperTime}(n) = n(\ln n)^{\theta}(\ln \ln n)^{4-\theta}$ with the same m, W_*, Δ and θ as in Theorem 1. `transform_main_order` additionally gives $D = \texttt{receipt}(n) < 8\, n^3$; the formula `receipt` differs from D_* in `eq:master-root` of the preprint but satisfies the required bound.

Cost model (`RAM.lean`). Every atom costs one unit of work: complex add/sub within a register kind; `scale` (scalar×scalar during preparation, or scalar×data); `inv` only on a scalar, with the nonzero condition in `valid`; the constants 0 and 1; integer literals and add/sub/mul/div/mod/lt; `len`/`look` and `id`/`fst`/`snd`. Combinators, iterations and recursion levels are charged additionally; building an array of length ℓ costs $1 + \ell$. There is no arbitrary complex constant and no atom converting between $\mathbb{C}$ and $\mathbb{N}$; branching uses integers only. The product of two data values (`cross`) is allowed only for convolution (`allow = true`); the DFT has none (`Prog false`). Inversion has its own record with `work` $= 1$ and `valid` $x \neq 0$, rather than a call of `Bill.one`. Types separate scalars from data.

Line-by-line comparison of definitions. We compared, line by line, the definitions (not the proof bodies): every feature of the model in `sec:model` of the preprint and every part of `thm:main` was matched with a concrete Lean construction (`release/lean/MODEL_AUDIT.md`; summary in Appendix C). The match is complete for all listed features: separate scalar and data registers (`Paint`); unit cost of every field operation; multiplication of the input only by prepared

scalars; division only on scalars with a nonzero denominator recorded in `valid`, not tested; no instruction producing an arbitrary complex constant (the root is an input); unit integer operations with polynomially bounded peak; a single supplied root of order $0 < D < 1024\,n^3$; exact complex arithmetic with no error field; no product of data for the DFT; and the time bound `paperTime` with natural logarithms. The chain of definitions of root and matrix is also confirmed: `zeta` $n = \exp(2\pi i/n)$, `fourierMatrix` $n\,j\,k =$ `zeta` n^{jk}, and `DFTProgram` requires `result.val` $=$ `dft` $n\,x = F_n x$. No weakening of the main goal was found.

Limits of the comparison. The adequacy of the `Bill` accounting with respect to a physical word-RAM remains a modelling judgement; there is no separate simulation theorem. We did not reread the proof bodies line by line; their correctness for these definitions is checked by the Lean kernel. Reproducing the compilation of someone else's formalisation is not an independent proof of the main theorem.

How to read the match. The formalisation uses a different, smaller network: 32 binary coordinates per factor, 4060 triples and the bound 2^{50}. The internal lemmas `lunch_big` and `alpha_theta` prove $W = O\big(n(\ln n)^{\alpha}(\ln\ln n)^2\big)$ with $\alpha = 1 - 2\cdot 10^{-11} < \theta$, from which the required upper bound `paperTime` follows. This stronger internal bound was checked separately with a short Lean statement assembled from these lemmas. Thus the *statement* of the main result is proved, but the concrete network of `network.tex`, its Δ and `net:tensor-bound` are not checked by this different construction. Synthesis and the general algebra of the appendix are not part of the formalisation; according to the introduction of the preprint this route is not needed for the stated exponent.

4.9 Software and environment

Python 3.12.3, numpy 1.26.4, mpmath 1.3.0, matplotlib 3.11.1, scipy 1.11.4 (a required dependency: `allength/compiler.py` uses `scipy.optimize.least_squares`). Intel Core i9-14900HX, 32 threads, 31 GiB RAM, Linux 7.0. The portable dependency set is `release/requirements-lock.txt` (installed into a clean virtual environment); the full freeze of the development environment is `release/requirements-full-freeze.txt`; see also `release/ENVIRONMENT.txt`.

4.10 Reproduction pipeline

All numbers in Section 5 come from the complete run `full-20261013-r3` of the pipeline `release/run_all.sh full`, which executed all 135 steps. Each run copies the sources, produces raw results, aggregates them into CSV files of that run only, validates every table of the manuscript against these CSV files, and regenerates the figures from the same CSV files; it records a manifest of commands, exit codes and hashes. The integrity of sources, logs, outputs, environment and the raw-to-CSV aggregation of this run was checked without substitution from archival data. Two exceptions are stated where they occur: the earlier timing measurement (Table 17) and the separate streaming $\mathbb{F}_p$ test at $r = 1024$, which is not one of the 135 steps. Availability is described in Section 8.

5 Results

5.1 Correctness of the full chain

Table 2 proves by computation that, for $r = 4, 7, 8, 9, 12, 15, 16$ and 32, the matrix of the exact builder `generic_word.py` equals $F_r(u)$; for $r = 4, 8$ and 16 this also holds for the C-slot path with

Table 2: Exact checks: every entry of the result is compared with u^{ij} for equality. The C-slot rows use the splitting (8) and conjugation via the DAG with inverse roots.

r	Field	Vectors	Entries checked	Mismatches
4	$\mathbb{Q}(\zeta_{32})$, degree 16	4 basis	16	0
7	$\mathbb{Q}(\zeta_{448})$, degree 192	7 basis	49	0
8	$\mathbb{Q}(\zeta_{64})$, degree 32	8 basis	64	0
9	$\mathbb{Q}(\zeta_{1152})$, degree 384	9 basis	81	0
12	$\mathbb{Q}(\zeta_{384})$, degree 128	12 basis	144	0
15	$\mathbb{Q}(\zeta_{1920})$, degree 512	15 basis	225	0
16	$\mathbb{Q}(\zeta_{128})$, degree 64	16 basis	256	0
32	$\mathbb{Q}(\zeta_{256})$, degree 128	32 basis	1024	0
9, 15, 16, 31, 32, 33, 63, 64, 127, 128	$\mathbb{F}_p$, three primes p	all basis	from 81 to 16 384 per prime	0
255, 256	$\mathbb{F}_p$, two primes p	3 random	765 and 768 per prime	0
512	$\mathbb{F}_p$, two primes p	2 random	1024 per prime	0
4, 8, 16 (C-slots + conjugation via DAG)	$\mathbb{Q}(\zeta_{32})$, $\mathbb{Q}(\zeta_{64})$, $\mathbb{Q}(\zeta_{128})$	all basis	16, 64, 256	0
16, 31, 32, 33, 64 (C-slots + conjugation via DAG)	$\mathbb{F}_p$	all basis	256, 961, 1024, 1089, 4096	0
128 (C-slots + conjugation via DAG)	$\mathbb{F}_p$	2 random	256	0

splitting and conjugation via the DAG. For larger r the exact checks in $\mathbb{F}_p$ are a necessary condition and agree with equality. The checked $\mathbb{F}_p$ widths are $r = 9, 15, 16, 31, 32, 33, 63, 64, 127, 128$ (not all integers up to 128; odd and prime widths included).

Streaming $\mathbb{F}_p$ test at $r = 1024$. A materialised $\mathbb{F}_p$ test at $r = 1024$ did not finish because of memory. A separate streaming test on 2026-10-14 (`stream_exact_dft.py`) completed: $N = 8192$, $p = 2305843009213800449$ and $p = 2305843009213931521$; the same two integer random vectors in each field, 2048 coordinates per field, 4096 in total, 0 mismatches. The generator reproduces the recursion of the strict word, applies each layer and discards it; the transposed pass reverses the order of layers and swaps the ends of shears. At small widths the streams were compared with the materialised word, and negative controls (wrong order, swapped ends, corrupted coefficient) were run. At $r = 1024$ each word has 4 796 928 layers and 64 639 488 gates. A repeat run took 431.9 s with peak RSS (GNU `time`) 37.91 MiB; its output and stream hashes coincide with the first run (450.95 s, 37.48 MiB). Source hashes were recorded at the start and at the end. This supplementary experiment is not one of the 135 steps; it tests a necessary condition on two vectors, not the whole matrix, complex equality or the C-slots at $r = 1024$.

For all widths the error decreases by about 100 decimal orders when the precision is raised by 100 digits; for the full DFT in Table 3 the decimal gain lies within 99.7–100.2 (for example, 99.7 at $r = 127$ and 100.1 at $r = 256$). This agrees with exact equality and reveals no systematic implementation error, but it is not a proof of equality for these r; only the exact checks of Table 2 provide that.

In Table 4 every shear block consists of 4 diagonal factors and 3 C-slots, and every shear of the word gives two blocks. For the conjugated coefficients we measure $\max_\mu |\bar{\mu}_{\mathrm{DAG}} - \mathrm{conj}(\mu)| / \max(1, |\mu|)$: this normalised deviation is $3.3 \cdot 10^{-89}$ at $r = 63$ with 100 digits and $1.2 \cdot 10^{-225}$ at $r = 128$ with 250 digits, and zero in some runs; it is not the absolute error of coefficients of large modulus. The C-slot path loses more digits than the shear word: by 1–3 digits at $r \leq 33$, about 6 at $r = 64$ (35.7 versus 30.0), about 13 at $r = 128$ (79.6 versus 67.0) and

Table 3: Full chain in mp at two precisions differing by 100 digits.

r	Vectors	Precision, digits	Max relative error	Digits lost	$\log_{10}\max_k\lvert H_k\rvert$	$\log_{10}\max_k\lvert 1/H_k\rvert$
7	7 basis	60 / 160	$1.5\cdot10^{-60}$ / $1.9\cdot10^{-160}$	0.2 / 0.3	0.9	0.1
8	8 basis	60 / 160	$2.5\cdot10^{-60}$ / $2.8\cdot10^{-160}$	0.4 / 0.5	1.0	0.1
9	9 basis	60 / 160	$4.2\cdot10^{-60}$ / $2.8\cdot10^{-160}$	0.6 / 0.4	1.1	0.2
15	15 basis	60 / 160	$5.2\cdot10^{-58}$ / $6.3\cdot10^{-158}$	2.7 / 2.8	1.6	0.5
16	16 basis	60 / 160	$1.4\cdot10^{-57}$ / $2.2\cdot10^{-157}$	3.1 / 3.3	1.7	0.5
31	31 basis	60 / 160	$1.0\cdot10^{-49}$ / $1.5\cdot10^{-149}$	11.0 / 11.2	2.9	1.4
32	32 basis	60 / 160	$1.9\cdot10^{-49}$ / $2.1\cdot10^{-149}$	11.3 / 11.3	3.0	1.5
33	33 basis	60 / 160	$1.5\cdot10^{-48}$ / $1.9\cdot10^{-148}$	12.2 / 12.3	3.1	1.6
63	63 basis	100 / 200	$1.8\cdot10^{-71}$ / $2.4\cdot10^{-171}$	29.3 / 29.4	5.3	3.5
64	64 basis	120 / 220	$9.1\cdot10^{-91}$ / $9.1\cdot10^{-191}$	30.0 / 30.0	5.4	3.6
96	3 random	200 / 300	$3.9\cdot10^{-153}$ / $6.5\cdot10^{-253}$	47.6 / 47.8	7.7	5.7
127	3 random	250 / 350	$9.8\cdot10^{-185}$ / $1.8\cdot10^{-284}$	66.0 / 66.3	10.0	7.9
128	128 basis	250 / 350	$1.1\cdot10^{-183}$ / $1.5\cdot10^{-283}$	67.0 / 67.2	10.0	7.9
256	3 random	450 / 550	$9.6\cdot10^{-309}$ / $7.7\cdot10^{-409}$	142.0 / 141.9	19.2	16.8

Table 4: The C-slot path with splitting and conjugation via the DAG, in mp (`mp_check_cslots.py --conj-dag`).

r	Vectors	Precision, digits	Shear blocks per vector	Max relative error	Digits lost
7	7 basis	60	84	$7.1\cdot10^{-60}$	0.9
9	9 basis	60	144	$2.2\cdot10^{-59}$	1.3
15	15 basis	60	8 260	$4.9\cdot10^{-57}$	3.7
31	31 basis	60	52 260	$8.2\cdot10^{-47}$	13.9
33	33 basis	60	59 232	$1.6\cdot10^{-45}$	15.2
63	63 basis	100	250 852	$1.3\cdot10^{-65}$	35.1
64	64 basis	120 / 220	258 944	$5.5\cdot10^{-85}$ / $7.1\cdot10^{-185}$	35.7 / 35.9
128	3 random	250 / 350	3 712 768	$4.3\cdot10^{-171}$ / $5.0\cdot10^{-271}$	79.6 / 79.7
256	1 random	450 / 550	20 205 056	$4.8\cdot10^{-281}$ / $2.4\cdot10^{-381}$	169.7 / 169.4

about 28 at $r = 256$ (169.7 versus 142.0). Two precisions in Table 4 exist only for $r = 64, 128$ and 256; for all three the error decreases by about 100 decimal orders, including $r = 256$ ($4.8\cdot10^{-281}$ at 450 digits to $2.4\cdot10^{-381}$ at 550 digits, a gain of 100.30 orders; digits lost 169.7 and 169.4). The run at $r = 256$ uses one random vector; a repeat measurement on 2026-10-14 at 550 digits took 448 s (7.5 min) to build the two words and 2326 s (39 min) in total. This is the time of our implementation; external load was not controlled.

5.2 Numerical instability

For the operator itself $F_r^* F_r = rI$, i.e. $\kappa_2(F_r) = 1$. The growth of the float64 error in Table 5 therefore cannot be explained by poor conditioning of the DFT: the computational path is unstable in finite precision. In exact arithmetic the same path gives the exact result (Table 2).

Conditioning of the factors. In Table 6 the second column is the numerically computed finite $\kappa_2(T)$, the third the finite $\kappa_2(D_H)$, and the fourth its asymptotic leading term cr with

$$c = \frac{\mathrm{Cl}_2(\pi/3)}{\pi \ln 10} = 0.14030576. \tag{18}$$

Table 5: Full chain in float64.

r	Vectors	Max rel. error	Median
8	8 basis	$9.1 \cdot 10^{-15}$	$2.8 \cdot 10^{-15}$
16	16 basis	$3.3 \cdot 10^{-12}$	$6.0 \cdot 10^{-13}$
24	24 basis	$3.4 \cdot 10^{-8}$	$8.0 \cdot 10^{-9}$
32	32 basis	$3.4 \cdot 10^{-4}$	$3.4 \cdot 10^{-5}$
48	6 random	$3.2 \cdot 10^{7}$	$8.7 \cdot 10^{6}$
64	6 random	$8.1 \cdot 10^{23}$	$6.7 \cdot 10^{23}$
96	6 random	$6.0 \cdot 10^{56}$	$3.7 \cdot 10^{56}$

Table 6: Conditioning of the triangular and diagonal factors (finite values) and the leading term cr.

r	$\log_{10}\kappa_2(T)$	$\log_{10}\kappa_2(D_H)$	Leading term cr
16	2.22	2.25	2.24
32	4.48	4.49	4.49
48	6.72	6.73	6.73
96	13.46	13.47	13.47
128	17.95	17.96	17.96
256	—	35.92	35.92
1024	—	143.673	143.673

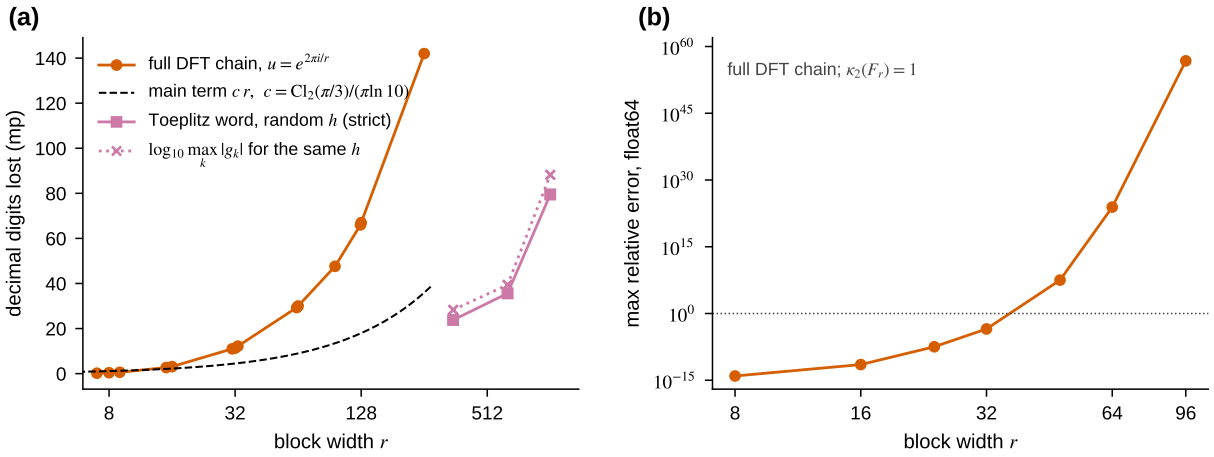


Figure 1: (a) Decimal digits lost in mp: full DFT chain (Table 3, all widths 7–256), Toeplitz word with random h (Table 8) together with $\log_{10}\max_k|g_k|$ for the same h, and the asymptotic leading term cr, $c = \mathrm{Cl}_2(\pi/3)/(\pi \ln 10)$. (b) Relative error of the full chain in float64 (Table 5); $\kappa_2(F_r) = 1$.

Here $\log_{10}\kappa_2(D_H) = \log_{10}\bigl(\max_k|H_k| / \min_k|H_k|\bigr)$ is an exact finite quantity (a difference of partial sums). Since $\log|H_k| = \sum_{s\le k}\log\bigl(2\sin(\pi s/r)\bigr)$, the difference between the maximum (at $k \approx 5r/6$) and the minimum (at $k \approx r/6$) is

$$\log_{10}\kappa_2(D_H) = \frac{r}{\pi}\,\frac{\mathrm{Cl}_2(\pi/3)}{\ln 10} + o(r), \tag{19}$$

where $\mathrm{Cl}_2(\pi/3) \approx 1.01494$ is the Clausen function (Proposition 2 in Appendix B.2). This is the leading asymptotic term, not an exact formula. The finite value differs from cr at the level $o(r)$: at $r = 16$ it is 2.25 against $cr = 2.24$, at $r = 1024$ it is 143.6732 against 143.6731 (difference 10^{-4}). In the Fourier case the conditioning of T is practically equal to that of D_H at the checked widths up to 128 (the largest and smallest singular values of T are mutually inverse to within 10^{-2}); we did not prove an exact formula for T. Hence the conditioning of the Toeplitz factor grows roughly like 10^{cr}, $c \approx 0.1403$; this is a property of the natural node order in the Newton form (Section 2.6), not of our implementation.

Stage-wise loss (Table 7 and Figure 2). (1) The diagonal stages lose almost no digits (≤ 2); the whole loss falls on the two Toeplitz words. (2) For T^{T} the local loss nearly equals the growth of intermediate values (5.6 and 6.5; 17.5 and 19.1; 42.4 and 43.9): the error behaves like ε times the largest intermediate quantity. (3) For T the local loss exceeds the growth; it is comparable with $\log_{10}\kappa_2(T)$ plus the growth (at $r = 64$: $9.0 + 9.2 \approx 18$ against 21.9), i.e. here

Table 7: Digit loss by stage (one random x). Local: the stage at working precision applied to the exact input; growth: $\log_{10}$ of the maximum modulus of an intermediate coordinate over the maximum modulus of the stage input.

r	Stage	Local loss	Accumulated loss	Growth	$\log_{10}$ output norm
32	D_H	0.5	0.5	3.0	2.5
32	T^{T}	5.6	5.6	6.5	4.1
32	D_{mid}	0.0	7.0	−0.4	3.6
32	T	6.7	9.4	3.0	1.9
32	D_H	0.2	10.9	−1.4	0.8
64	D_H	0.3	0.3	5.4	4.9
64	T^{T}	17.5	18.0	19.1	8.9
64	D_{mid}	−0.3	20.9	−0.7	8.1
64	T	21.9	26.5	9.2	4.0
64	D_H	0.3	29.1	−3.5	0.9
128	D_H	2.0	2.0	9.8	9.4
128	T^{T}	42.4	42.3	43.9	17.7
128	D_{mid}	1.5	48.6	−1.0	16.8
128	T	50.7	59.8	21.7	8.4
128	D_H	1.9	66.3	−7.8	1.1

the amplification of the input error by the operator itself is added. (4) The total loss (Table 3) amounts to 2.5, 3.3, 3.7 and 4.0 units of $\log_{10} \kappa_2(T)$ at $r = 32, 64, 128, 256$.

Where the growth arises (Table 14 in Appendix A). The growth decomposition of Section 4.5 describes growth, which can amplify rounding in T^{T} (where the local loss nearly equals the growth); for T the loss exceeds the growth, so there the decomposition describes growth only. Its sums coincide with the growth column of Table 7 (T^{T}: 6.5, 19.1, 43.9; T: 3.0, 9.2, 21.7). (1) All of the record-maximum growth of T^{T} occurs in the cross-block chunk phase, i.e. in the replay of displacement DAGs built from the reciprocal series g, not in leaf or direct layers. For T the chunk phase gives 67–84%, the rest the direct triangular word of base blocks of width 8 (below $v = 10$ there is no admissible chunk, Section 3.4). (2) The growth is distributed over several recursion levels rather than concentrated in one. In T^{T} the outer level $v = r$ contributes most (49–66%); in T the contribution shifts to middle levels. A zero increment at some level (for example $v = 16$ for T^{T} at $r = 128$) means only that no new record occurs there, not that the level does not contribute to the error. (3) Hence the large intermediate quantities are localised in the displacement replay of chunks (growth of the reciprocal series g, Section 5.3) and accumulate over recursion levels.

Phase-precision control (Table 15 in Appendix A). Raising the precision of the chunk phase alone reduces the loss of T^{T} by about 14.5 and 37.5 digits at $r = 64$ and 128; raising the precision of the direct phase alone has almost no effect. For T the reduction is about 10.3 and 27.9 digits, but a residual loss of 11.6 and 22.6 digits remains. Thus, for the checked inputs and widths, the precision of the chunks strongly affects the error of both stages, especially of T^{T}. This is not an additive decomposition of the error: rounding in the phases and subsequent amplification interact, and the remainder cannot be called mere propagation without a separate control. The experiment covers one random vector (seed = r) per width, the fixed DFT coefficients, and $r = 64, 128$; it does not cover other coefficient distributions or wider cases.

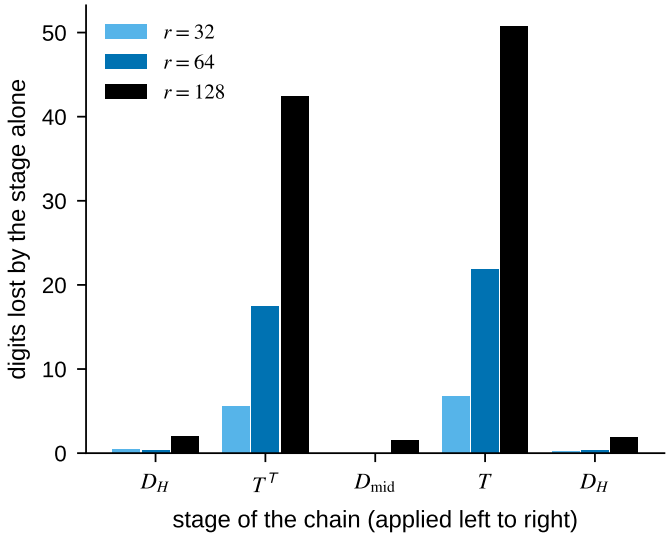


Figure 2: Local digit loss by stage of the chain D_H, T^{T}, D_{mid}, T, D_H at $r = 32, 64, 128$ (Table 7).

Table 8: Toeplitz word, random h, strict mode.

r	Precision, digits	Max relative error	Digits lost	$\log_{10} \max_k \lvert g_k \rvert$
352	300 / 400	$5.8 \cdot 10^{-277}$ / $5.5 \cdot 10^{-377}$	23.8 / 23.7	28.3
640	400 / 500	$4.4 \cdot 10^{-365}$ / $5.6 \cdot 10^{-465}$	35.6 / 35.7	39.4
1024	600 / 700	$3.5 \cdot 10^{-521}$ / $1.7 \cdot 10^{-621}$	79.5 / 79.2	88.3

5.3 Toeplitz word with random coefficients

This is a test of the general Toeplitz operator Tx, not of the DFT, run in strict mode with three vectors and two precisions (Table 8). For comparison, the zero-skipping variant with one vector lost 23.6, 35.0 and 78.6 digits at $r = 352, 640, 1024$.

Mechanism of coefficient growth. The series g is the reciprocal series of h (`loc:reciprocal`) from which the chunks are built; its radius of convergence equals the smallest modulus ρ of a root of $h(z) = \sum_k h_k z^k$. Hence $\log_{10}|g_k|$ grows with slope $\log_{10}(1/\rho)$.

The slope coincides with $\log_{10}(1/\rho)$ to the fourth decimal, and the prediction $(r-1)\log_{10}(1/\rho)$ differs from the observed maximum by less than one digit (Table 9). About half of the roots lie inside the unit disc; the smallest modulus is 0.82–0.86. Thus, for these three samples, the growth of g is explained by the smallest root of $h(z)$, and the digit loss nearly equals $\log_{10} \max_k |g_k|$. We did not study the distribution of ρ for other coefficient distributions.

Table 9: Roots of $h(z)$ and growth of g (`roots_h.py`; roots are eigenvalues of the companion matrix, relative residual $|h(z)|$ at the smallest roots $\le 4 \cdot 10^{-14}$). The slope is fitted on the second half of k.

r	$\rho = \min\lvert\text{root}\rvert$	Roots inside unit disc	Slope of $\log_{10}\lvert g_k \rvert$	$\log_{10}(1/\rho)$	$(r-1)\log_{10}(1/\rho)$	Observed $\log_{10} \max_{k<r} \lvert g_k \rvert$
352	0.8256	179 of 351	0.0832	0.0832	29.2	28.3
640	0.8647	314 of 639	0.0632	0.0632	40.4	39.4
1024	0.8185	508 of 1023	0.0870	0.0870	89.0	88.2

Table 10: Strict ASAP depth: adaptive rule versus chunk size 1. The last column is the depth ratio of the value-dependent zero-skipping variant (archival data).

r	b	Adaptive	Chunk 1	Adaptive / chunk 1	Zero-skipping variant, adaptive / chunk 1
64	1	37668	37668	1.00	—
128	2	133924	152356	0.88	—
256	2	518948	611108	0.85	1.29
288	2	656793	773433	0.85	1.29
320	2	811515	955515	0.85	1.29
336	3	626622	1053510	0.59	0.89
352	4	497926	1156166	0.43	0.64
384	4	592581	1375941	0.43	0.64
512	4	1035044	2446116	0.42	0.65
640	4	1617915	3822715	0.42	0.65
768	4	1753797	5504709	0.32	0.49
1024	8	1682212	9786148	0.17	0.27

5.4 Fixed topology

For every run we compute SHA-256 of the sequence of layers, each layer recorded as its ASAP round and its list of gates (type M for scaling or S for shear, source, destination). The audit covers 12 widths, 2 rules (adaptive and chunk size 1) and 3 inputs (seed $= r$, another seed, $h =$ ones); the complete data are in Table 18 (Appendix A). For every width and rule the hash is identical for all three inputs. Hence, in strict mode, the gate types, the coordinate pairs, the order of layers, the schedule by rounds, the depth and the gate count are functions of the width alone. Together with the static audit of Section 3.6, this confirms the fixed-topology requirement of the preprint for our implementation. For comparison, in the zero-skipping variant the depth at $r = 64$ was 24442 and 24452 for two seeds and 13969 for $h =$ ones.

5.5 Depth

The strict ASAP depth was measured on a grid of 12 widths: 64, 128, 256, 288, 320, 336, 352, 384, 512, 640, 768, 1024 (`strict_tables.py`; data in `paper/data/strict_tables.csv`).

In strict mode the adaptive rule has depth not larger than chunk size 1 at all 12 widths of the grid, and strictly smaller from $r = 128$ on (Table 10 and Figure 3). The crossover between 320 and 336 seen in the zero-skipping variant disappears: there chunk size 1 gained an advantage because in the DAGs for chunks of size 1 many pointwise multipliers are exactly zero ($g(1) = 6$ instead of 8, $L(1) = 18$ instead of 28). The jump of the ratio between 320 and 336 coincides with the jump of b from 2 to 3 and persists in both modes. These conclusions concern this grid; we do not claim them for intermediate r. The depth is not monotone in r: at $r = 352$ it is smaller than at $r = 320$, because the larger block admits a larger chunk.

Depth model. We use the recurrence

$$d(v) = d(\lfloor v/2 \rfloor) + k^2 L(b), \qquad k = \lceil \lfloor v/2 \rfloor / b \rceil, \tag{20}$$

with direct cost $v(v+1)/2$ for blocks without an admissible chunk; b follows the strict table of $g(b)$, and $L(b)$ is the number of replay layers of one $b \times b$ pair in strict mode (`depth_exact_rule.py --strict`). The model uses the measured $g(b)$ and $L(b)$ and follows only the branch $\lfloor v/2 \rfloor$; for powers of two both branches coincide.

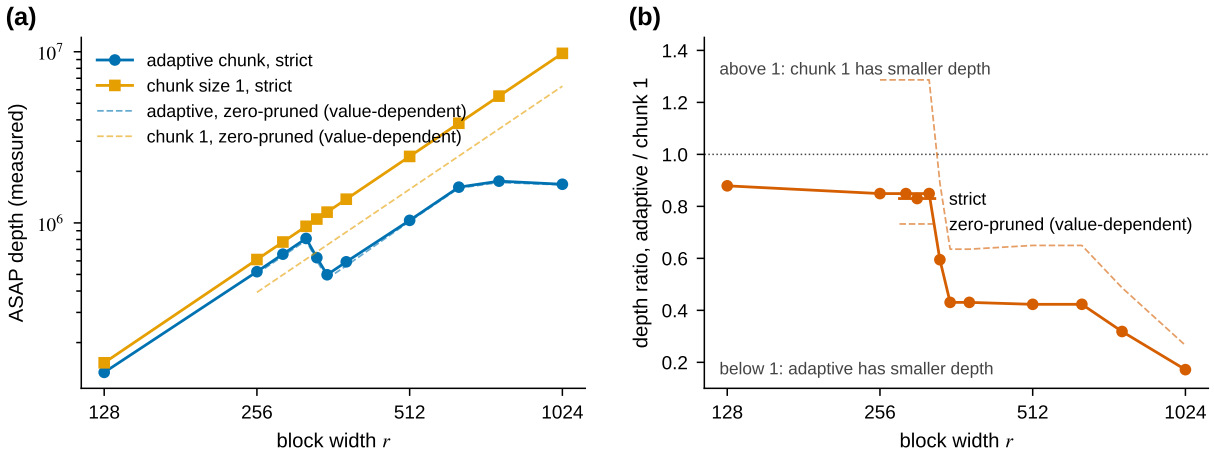


Figure 3: (a) Measured ASAP depth of the strict word for the adaptive rule and chunk size 1 (solid lines) and the same quantity for the value-dependent zero-pruned variant (thin dashed lines, archival data). Logarithmic axes. (b) Depth ratio adaptive / chunk 1; dotted line: equality. Below 1 the adaptive rule has smaller depth. In strict mode the ratio is below 1 at all widths from 128 on; the crossover of the zero-pruned variant near $r \approx 330$ is an artifact of zero skipping. Data: Table 10.

Table 11: Strict depth model versus measured ASAP depth (adaptive rule).

r	Model	ASAP	ASAP / model
64	37668	37668	1 (exact)
128	133924	133924	1 (exact)
256	518948	518948	1 (exact)
288	656793	656793	1 (exact)
320	811515	811515	1 (exact)
336	626207	626622	1.00066
352	497783	497926	1.00029
384	592581	592581	1 (exact)
512	1035044	1035044	1 (exact)
640	1617915	1617915	1 (exact)
768	1753797	1753797	1 (exact)
1024	1682212	1682212	1 (exact)

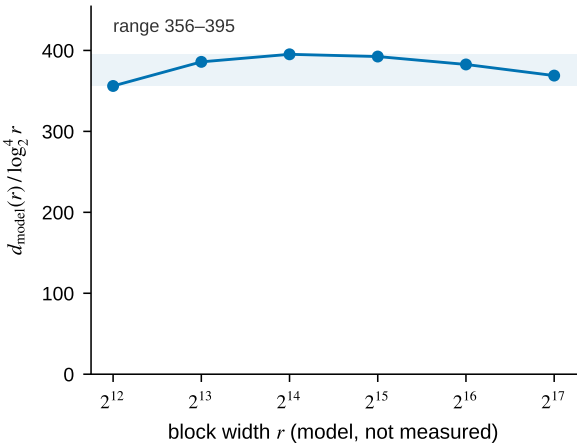


Figure 4: Ratio $d_{\text{model}}(r)/\log_2^4 r$ of the strict depth model at powers of two 2^{12}–2^{17}; the band marks the range of values. Model values, not measurements. Data: Table 16.

The model matches the measurement exactly at 10 of 12 widths (Table 11). At $r = 336$ and 352 the recursion reaches odd blocks $v \geq 10$ (21 and 11), and the model, which follows only the branch $\lfloor v/2 \rfloor$, underestimates the depth. This directly confirms that the full recurrence must take the maximum over both branches, as in the preprint's proof (6).

At powers of two 2^{12}–2^{17} the model gives $d_{\text{model}}/\log_2^4 r$ within 356–395 (Table 16 in Appendix A, Figure 4). These values come from the model; we did not build ASAP words of these widths. Because the model is built on the empirical $g(b)$ and $L(b)$, its agreement with the measurements is a consistency check, not an independent proof of $O(\log^4 r)$. The order-of-magnitude heuristic

$$g(b) \approx C\, b \log b \;\Rightarrow\; b = \Theta(v/\log v), \quad k = \Theta(\log v), \quad L(b) = \Theta(\log v), \tag{21}$$

so that one level contributes $\Theta(\log^3 v)$ over $\log_2 r$ levels, explains why the model behaves like $\log^4 r$ and matches the preprint's proof of the upper bound. Six nearly constant points are consistent with $O(\log^4 r)$ but do not establish an asymptotic equality.

Table 12: Real flops. Ratios are DFT via the word over radix-2 FFT, and one Toeplitz word over circulant-FFT and direct Toeplitz multiplication.

r	One Toeplitz word	DFT via word	Radix-2 FFT	Toeplitz via circulant FFT	Direct Toeplitz	DFT via word / FFT	Word / circulant FFT	Word / direct
64	518 272	1 037 696	1 920	9 728	16 640	540	53	31
128	7 426 304	14 854 912	4 480	22 016	66 048	3 316	337	112
256	40 411 648	80 827 904	10 240	49 152	263 168	7 893	822	154
512	157 107 200	314 223 616	23 040	108 544	1 050 624	13 638	1 447	150
1024	517 113 856	1 034 246 144	51 200	237 568	4 198 400	20 200	2 177	123

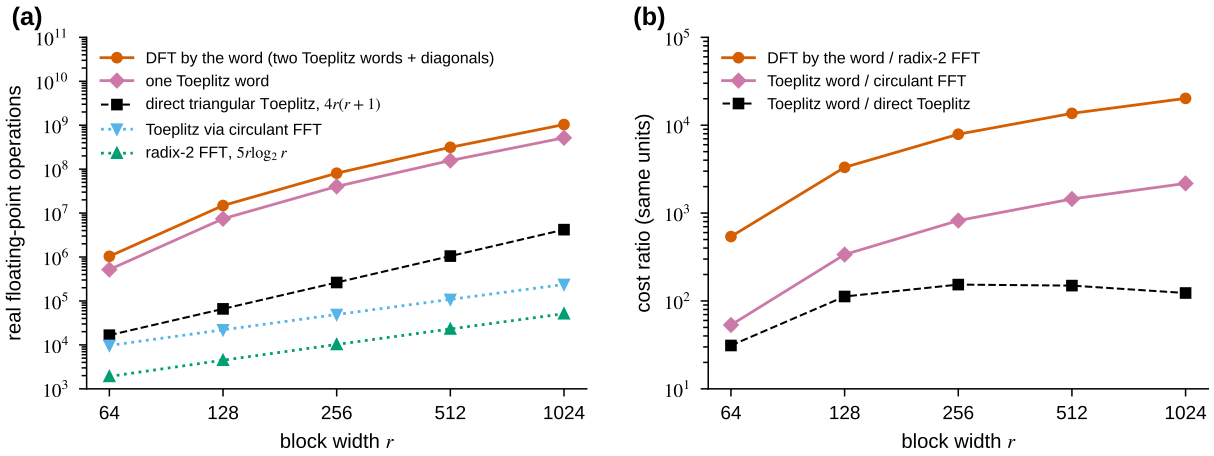


Figure 5: (a) Real flops of the DFT via the word, one Toeplitz word, direct Toeplitz multiplication, Toeplitz multiplication via circulant FFT, and radix-2 FFT. (b) Cost ratios in the same units. Data: Table 12.

5.6 Cost in identical units

Table 12 gives real flops (strict mode, adaptive rule, units of Section 4.7).

The gate count of one word divided by r^2 is 15.8, 56.7, 77.1, 74.9 and 61.6 at $r = 64, \ldots, 1024$. This quantity is not constant, so the data do not establish a quadratic cost law. At the checked widths the word is more expensive even than direct multiplication (Figure 5). This is the cost of stand-alone execution of the word; the preprint does not use the word this way and claims no advantage at these widths (Section 4.7). The preprint cites Alman and Rao [3] with leading term $(15/4)\, n \log_2 n$ real operations for the DFT at powers of two; against this reference the gap would be even larger.

Gate–depth trade-off at $r = 512$ (Figure 6). Strict mode, three rules:

- adaptive ($b = 4$): depth 1 035 044, 19 638 528 gates;
- chunk size 1: depth 2 446 116, 4 647 168 gates;
- cap2 ($b \leq 2$): depth 2 059 044, 22 882 560 gates.

The adaptive rule and chunk size 1 form a trade-off: chunk size 1 has 4.2 times fewer gates and 2.4 times larger depth. cap2 is worse than the adaptive rule in both measures, i.e. it is dominated among these three rules. Only three rules were checked. Other chunk sizes, different rules at different recursion levels, or another schedule could give better points, so we do not claim that no rule minimises both quantities.

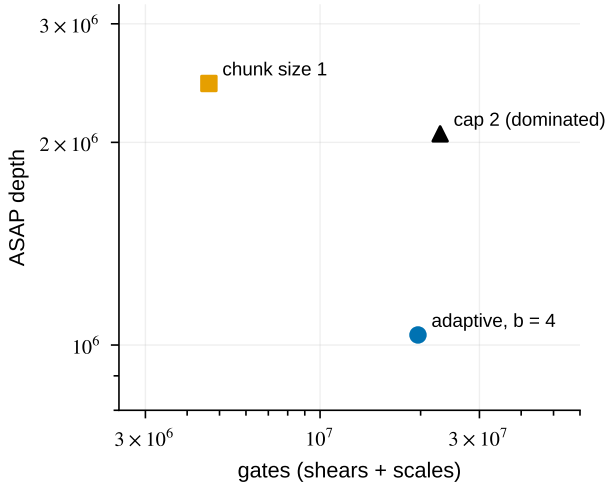


Figure 6: Gates and depth at $r = 512$ for three rules in strict mode. Both axes logarithmic. There is no spread over inputs, because the topology is fixed.

Table 13: Python implementation (`timing.py`), measurement of 2026-10-14 without parallel steps of the run. Build: preparation of scalars and schedule.

r	Gates	Build of word, s	Apply to one vector, s	Peak memory, GiB	numpy.fft of length r, µs	Dense Toeplitz product (numpy), µs
64	64 800	0.13	0.01	0.04	0.8	2.0
128	928 320	0.73	0.13	0.12	1.2	6.1
256	5 051 520	3.5	0.71	0.50	1.7	22.0
512	19 638 528	13.2	2.8	1.79	2.7	82.9

5.7 Rounds and conflicts

In strict mode (`rounds_check.py`) execution by rounds gives the same result as sequential execution: the difference is 0.0 at $r = 16, 32, 48, 64$. Within a round there are no coordinate conflicts: 0 at $r = 256$ (779 392 layers, 518 948 rounds, 1.50 layers per round) and at $r = 512$ (2 074 880 layers, 1 035 044 rounds, 2.00 layers per round).

5.8 Time and memory of our implementation

One process per width, float64, strict mode, adaptive rule, the machine of Section 4.9, command `timing.py r`. We measure one Toeplitz word on random complex h and x (`numpy.default_rng(r)`, real and imaginary parts `standard_normal`, h_0 replaced by $|h_0| + 1$), not the full DFT chain with two words and diagonal factors. The code labels RSS as GB, but `ru_maxrss`/1024^2 on Linux gives GiB; the tables use GiB.

The measurement of Table 13 was made on 2026-10-14 in the run `full-20261013-r3`. The run executes its steps sequentially, so the steps `timing-64`, ..., `timing-512` did not overlap with other steps of the run; each is a separate process in a clean virtual environment (Python 3.12.3, numpy 1.26.4) with `OPENBLAS_NUM_THREADS` = `OMP_NUM_THREADS` = `MKL_NUM_THREADS` = 1. Other processes on the machine were not controlled. The four steps together took 24 s. An earlier measurement with the same command, taken while seven other single-threaded computations (of 32 threads) ran on the machine, is kept unchanged as a record of those conditions in Table 17 (Appendix A).

In these timing runs the intermediate values overflow in float64 for $r \geq 256$, so the outputs are not used as a correctness check; the time measures the execution of the same fixed gate list in Python, while exact correctness and numerical behaviour were checked by the separate

experiments above. The gate counts are identical in both measurements and equal the sum of shears and scalings of the adaptive word in Table 18. The ratio of the repeat time to the earlier one is 0.81–0.88 for the build, 0.81–1.00 for the application and 0.73–0.81 for `numpy.fft`. Building the word in mp is more expensive: 7–83 s at $r = 127$–256 with 250–450 digits in the earlier logs (Table 3) and 3–79 s in the repeat run. These numbers describe our Python implementation on this machine, not the algorithm of the preprint. They show the order of magnitude: our implementation applies one word roughly 10^4 ($r = 64$) to 10^6 ($r = 512$) times slower than `numpy.fft` performs the whole transform (both in the repeat and in the earlier measurement), and the memory grows with the gate count. At $r = 512$ the word is built in 13.2 s (15 s in the earlier measurement).

6 Discussion

6.1 The exponent and the full formula

With δ from (2), the ratio of the leading factors is

$$\frac{n(\log n)^{\theta}(\log\log n)^{4-\theta}}{n\log n} = \frac{(\log\log n)^{3+\delta}}{(\log n)^{\delta}}. \tag{22}$$

It tends to zero, so the claimed bound is indeed $o(n\log n)$. At practical n, however, it is large. At $n = 10^{300}$ with natural logarithms ($\ln n \approx 690.8$, $\ln\ln n \approx 6.54$) the ratio is ≈ 279.446, ignoring hidden constants (with $\log_2$ it would be ≈ 988). The ratio drops below 1 only when

$$\delta\,\ln\ln n > (3+\delta)\,\ln(\ln\ln n), \qquad \text{i.e. at } \ln\ln n \approx 4.81494\cdot 10^{14}. \tag{23}$$

This is not a forecast of running time, but an explanation of why a small difference of exponents does not mean a practical speed-up. The numerical value of θ was recomputed in Section 2.2; our finite local experiments do not test the claimed global asymptotic time bound, which requires an analytical or formal proof.

6.2 The scale gap

$m = 10^6$ and $W_* = 2^{71}$ are parameters of the tensor network, not widths of the local compiler. The local widths in the proof are the first ℓ odd primes and one power of two $2^e < 2r_{\ell+1}$, $\ell = \Theta(\log n/\log\log n)$ (`all-lengths.tex:13--56`); widths of these types (odd primes such as 7, 31 and 127, and powers of two) are partly covered by our tests. The concrete network recursion applies only in sectors with $k \geq K = m(71+1) = 72\,000\,000$ axes (`network.tex:494--498`), and $k \leq \ell$ (`synchronization.tex:82`). Then $\ell \geq 72\,000\,000$, and already the elementary lower bound $r_\ell \geq \ell + 2$ shows that the largest local width exceeds 1024 by a factor of more than 70 thousand. This compares the largest width; it does not claim that all axes are large. Extrapolating our numerical behaviour to these widths is not justified.

6.3 Work versus time

The preprint bounds the full time, including scalar preparation and index work. We count gates of the word and their flops, and measure time and memory only for our implementation (Tables 13 and 17).

6.4 Value-dependent optimisations

Skipping zero multipliers does not change the operator, so it is admissible for a numerical implementation of the operator. But it lies outside the construction of the preprint, since it needs a zero test on a complex coefficient that the construction does not use (`local.tex:42`),

and it changes the object being measured: depth and work become properties of the pair (width, coefficients) rather than of the width. The methodological lesson is direct: a value-dependent variant can produce structural effects that do not exist in the construction under test. In our case it produced a spurious depth crossover between $r = 320$ and 336 and a spurious seed-to-seed spread of the depth (Section 3.6); all results reported here use the strict construction. Degenerate inputs ($h = \text{ones}$) are interesting in themselves and should be analysed separately rather than excluded; in strict mode they give the same structure as generic ones.

6.5 Numerical instability

The construction of the preprint is designed for exact arithmetic, and the preprint claims no stability. Our data show how much this restriction matters. In the Fourier case the conditioning of the Toeplitz factor grows roughly like 10^{cr} with $c = \mathrm{Cl}_2(\pi/3)/(\pi \ln 10) \approx 0.1403$ (the asymptotic leading term of the finite conditioning of D_H), and the full chain loses 3–4 such units, i.e. about $0.5\,r$ decimal digits (142 at $r = 256$). In float64 the chain is unusable already from $r \approx 32$. The ordinary FFT is stable. Direct execution of this word (with the natural node order) in finite precision would require, at the checked widths, a number of digits growing roughly linearly with r; this changes the bit cost. The preprint does not claim anything about bit complexity (`introduction.tex:101`), so this is not a contradiction but a quantitative illustration of a limitation it names itself. The conclusion concerns only the local component: it does not mean that the full algorithm is unstable in every implementation, and it does not establish a lower bound on the cost of other modifications.

6.6 Lower bounds

The bounds of Morgenstern and Ailon concern bounded coefficients, unitary gates or bounded conditioning. Our data show that the local compiler indeed lies outside these frameworks: the conditioning of its factors grows exponentially with r. This is consistent with the bounds not applying to it, but says nothing about the correctness of the network.

6.7 Implications for the preprint

Within the checked range the local component behaves as claimed: the implemented operator equals $F_r(u)$ (proved by computation in $\mathbb{Q}(\zeta_N)$ for eight widths 4–32; consistent with equality for the other checked r), the topology of the strict word depends only on r, and the depth model is compatible with $O(\log^4 r)$. We found no discrepancies with the claims of the preprint. This neither confirms nor refutes the main theorem, whose effect comes from the network. Checking the proof of the preprint itself requires an audit of the network and of the synchronisation; the comparison of the Lean definitions is complete within the scope of Section 4.8. Our measurements quantify, for the local component, limitations that the preprint names itself (enormous constants, exact arithmetic; `introduction.tex:67`, `:101`) and agree with the practical remarks of Recht [11]; they say nothing about the correctness of the preprint.

7 Limitations and threats to validity

Scope of the checks. The full DFT is proved by computation in $\mathbb{Q}(\zeta_N)$ at eight widths from 4 to 32; in $\mathbb{F}_p$ (necessary condition) it is checked for the whole matrix at ten selected widths from 9 to 128, for a few vectors at $r = 255, 256, 512$, and separately in streaming mode at $r = 1024$ (the same two vectors in two fields); in mp at $r \le 256$. The C-slots are proved by computation in $\mathbb{Q}(\zeta_N)$ at $r = 4, 8, 16$, checked in $\mathbb{F}_p$ for the whole matrix at $r = 16, 31, 32, 33, 64$ and by sampling at $r = 128$, and in mp at $r \le 256$. The Toeplitz word with random h is checked at $r \le 1024$; depth is measured at $r \le 1024$ and modelled up to $r = 2^{17}$. In mp at $r = 96, 127, 256$

and in $\mathbb{F}_p$ at $r = 255, 256, 512$ a few random vectors were checked rather than the whole matrix; in $\mathbb{F}_p$ at $r = 127$ all basis vectors were checked.

Strength of evidence. Exact checks in $\mathbb{F}_p$ are a necessary condition, not a proof of the complex equality; only the checks in $\mathbb{Q}(\zeta_N)$ are proofs by computation. Agreement in mp is numerical agreement only.

Depth. The ASAP depth is a particular schedule, neither an optimum nor a lower bound. The equivalence of our chunk rule with the preprint's is established only for $b \leq 1024$ (Section 3.3). The depth model uses the empirical $g(b)$ and $L(b)$ and follows only the branch $\lfloor v/2 \rfloor$.

Instability analysis. Stage-wise measurements describe the observed distribution of digit loss (Table 7); the growth decomposition describes record-maximum increases over recursion levels and phases (Table 14); and the phase-precision control (Table 15, $r = 64, 128$) measures the sensitivity of the error to the precision of the cross-block chunks for the checked inputs. Together, these diagnostics do not provide a complete causal decomposition of rounding error. This control does not cover coefficient preparation, and for T a significant loss remains after it. Not done: a decomposition down to individual chunks, a control for other coefficient distributions with possible cancellation, and widths beyond $r = 128$.

Cost. Scalar preparation, indices and memory were measured only for our Python implementation; flops are counted by the standard scheme without special coefficients.

Lean. We rechecked the closure of the main goal, the axioms, and the correspondence of the definitions `DFTGoal`/`ConvGoal`/`RAM` with the statements of the preprint. The formalisation proves the required bound through a different network; `network.tex`, Δ and `net:tensor-bound` are not checked by it. We did not reread the proof bodies; the correspondence of `Bill` with a physical word-RAM remains a modelling judgement; there is no separate simulation theorem. This is a reproduction of someone else's mechanical check, not our own independent proof.

Main theorem. The numerical value of θ was recomputed (Section 2.2), but the main asymptotic saving, the network, the synchronisation, the algorithm for lengths of all sizes and the synthesis were not checked by local experiments.

Threats to validity. The exact checks are executed by `generic_word.py`, whereas the numerical ones use `mp_exact`/`mp_compiler` and `conv_dag`/`rounds`. These are separate implementations of one described construction, so equality in an exact field applies directly to the exact checker. An additional comparison of the sequences of layers and coordinate pairs (without coefficients) showed that the three strict builders coincide at $r = 4, 7, 8, 9, 12, 15, 16, 31, 32, 33, 64, 128$ for $h_k = 1/(k+1)$; this supports structural correspondence but is not a proof that their coefficients coincide, nor a statement for all widths. Our own shear decomposition, the early stopping of the chunk rule and the ASAP schedule are implementation choices; their checked limits are stated above. The phase and random-Toeplitz diagnostics use one seed per width; no broader statistical generalisation is claimed. Timings depend on the machine and on concurrent load.

8 Data and code availability

The code, the data, and the raw logs of the complete run `full-20261013-r3`, together with its manifest and hashes, will be available at `https://github.com/cyberdid/fft-power-saving-verification` under the Apache License 2.0. Paths below are relative to the root of that repository.

- `release/README_REPRODUCE.md`: reproduction steps and the mapping from scripts to tables.
- `release/run_all.sh quick|full`: an isolated run, i.e. a copy of the sources, then raw results, then the CSV files of this run, then validation of all tables of the manuscript against them, then the figures from the same CSV files. Every run has its own directory and a manifest of commands, exit codes and hashes. Mode `full` executes all 135 steps; mode `quick` explicitly skips the six longest experiments and therefore does not confirm a full reproduction. An interrupted run can be resumed in the same environment; resumption checks that the captured sources, the environment and the results of completed steps are unchanged.
- `release/evidence/full-20261013-r3/`: the raw logs, aggregated data, figures, environment freeze and manifest of the complete run used in this paper; the manifest records the SHA-256 of every captured source file, of every log and of every output. The files are byte-identical to those of the run and therefore keep the absolute paths of the machine on which it was executed. The comparison curves of the zero-pruned variant in Figure 3 are archival and labelled as such.
- `paper/data/supplemental/fp1024/`: data, logs and reproduction instructions of the separate streaming $\mathbb{F}_p$ test at $r = 1024$ (`paper/scripts/stream_exact_dft.py`), which is not one of the 135 steps.
- `release/requirements-lock.txt`: pinned direct and transitive dependencies, installed into a clean environment with a successful `pip check`; `release/requirements-full-freeze.txt`: the full freeze of the development environment (a record, not a lock file); `release/ENVIRONMENT.txt`.
- `release/lean/`: `lean-toolchain`, `lakefile.lean`, `lake-manifest.json`, `AxiomCheck.lean`, `MODEL_AUDIT.md` and the build logs.
- `release/SHA256SUMS`: checksums of the code, data and figures of this version; it certifies the integrity of the recorded files, not the proof of any theorem.

The code is in `allength/` (among others `conv_dag.py` with the flag `KEEP_ZERO_MULTIPLIERS`, `mp_exact.py` with `ROOT_SIGN`, `exact_field.py`, `generic_word.py`, `rounds.py`, `mp_compiler.py`, `compiler.py`, `depth_recurrence.py`); scripts are in `paper/scripts/`, aggregated data in `paper/data/*.csv`. The preprint is available in the repository `https://github.com/openai/math` (directory commit 301488868, HEAD fd4aeeb).

9 Conclusion

We independently checked the local compiler of OpenAI's preprint in its strict form, in which the topology depends only on the width, and confirmed this property by a code audit and by structure hashes. For eight widths from 4 to 32 we proved by computation in cyclotomic fields that the matrix of the exact builder equals $F_r(u)$. For selected widths up to $r = 512$, and in a separate streaming test at $r = 1024$, exact checks in finite fields give a necessary condition (by sampling at $r = 255, 256, 512, 1024$), and high-precision arithmetic up to $r = 256$ agrees with equality. The C-slot path (our shear decomposition; the splitting and the conjugation via the DAG with inverse roots as in the preprint) is proved by computation at $r = 4, 8, 16$.

In finite precision this exact construction is strongly unstable; the preprint does not claim stability, so this contradicts none of its statements. The conditioning of the Toeplitz factor in the Fourier case grows roughly like 10^{cr}, $c \approx 0.1403$ (the asymptotic leading term of the finite conditioning, through the Clausen function); the full chain loses about $0.5\,r$ decimal digits,

almost all of them in the two Toeplitz words, and is unusable in float64 already from $r \approx 32$, although $\kappa_2(F_r) = 1$. The measured depth of the strict word up to $r = 1024$ and the depth model up to 2^{17} are compatible with the upper bound $O(\log^4 r)$. The cost of stand-alone execution of the word in real flops is two to four orders of magnitude above the FFT at the checked widths, and above even direct multiplication; the preprint does not use the word in this way (Section 4.7). A variant that drops exactly-zero multipliers is value-dependent and produces depth effects absent from the strict construction, which is why all results here use the latter.

Our local experiments do not test the main asymptotic saving with θ. The Lean formalisation of OpenAI that we recompiled proves `DFTGoal` with the required upper bound through a different network; the concrete `network.tex` and its Δ are not checked by it. The comparison of definitions is complete within the scope of Section 4.8; the adequacy of `Bill` with respect to a physical RAM remains a modelling judgement, and we did not reread the proof bodies line by line. Our results neither refute the theorem nor constitute an independent proof of its main effect; they delimit the checked local implementation and the limits of its use in finite precision.

References

[1] Nir Ailon. A lower bound for Fourier transform computation in a linear model over 2×2 unitary gates using matrix entropy. arXiv:1305.4745, 2013.

[2] Nir Ailon. An $n \log n$ lower bound for Fourier transform computation in the well conditioned model. arXiv:1403.1307, 2014. v1 2014-03-06; title of the current version v5, 2014-07-24. The preprint's bibliography cites the earlier title "An $\Omega((n \log n)/R)$ lower bound for Fourier transform computation in the R-well conditioned model".

[3] Josh Alman and Kevin Rao. Faster Walsh–Hadamard and discrete Fourier transforms from matrix non-rigidity. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing (STOC 2023)*, pages 455–462. ACM, 2023. doi: 10.1145/3564246.3585188. Extended version: arXiv:2211.06459, 2022-11-11.

[4] James R. Bunch. Stability of methods for solving Toeplitz systems of equations. *SIAM Journal on Scientific and Statistical Computing*, 6(2):349–364, 1985. doi: 10.1137/0906025.

[5] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1.

[6] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, Philadelphia, second edition, 2002. doi: 10.1137/1.9780898718027.

[7] Thomas Kailath, Sun-Yuan Kung, and Martin Morf. Displacement ranks of a matrix. *Bulletin of the American Mathematical Society (New Series)*, 1(5):769–773, 1979. doi: 10.1090/S0273-0979-1979-14659-7.

[8] Jacques Morgenstern. Note on a lower bound on the linear complexity of the fast Fourier transform. *Journal of the ACM*, 20(2):305–306, 1973. doi: 10.1145/321752.321761.

[9] OpenAI. An explicit power saving for the exact discrete Fourier transform. Preprint, 2026-09-25, OpenAI Math repository, `https://github.com/openai/math`, September 2026. Directory `preprints/An-explicit-power-saving-for-the-exact-discrete-Fourier-transform-September-25-2026` (`main.pdf`; sources `build/main.tex`, `build/sections/*.tex`, `build/references.bib`). Last change of the directory: commit 301488868, 2026-10-07; repository HEAD fd4aeeb. The Lean formalisation `lean/OAI/Computability/FourierTransform` (scope described in `lean/docs/130.md`) is a separate artifact of the same repository.

Table 14: Decomposition of the growth of intermediate quantities by levels and phases (one random x, seed $= r$; growth in $\log_{10}$). The last column lists the increments by block width v.

r	Stage	Growth	In chunk phase	In direct word	Distribution over levels v (increment)
32	T^{T}	6.5	100%	0%	v=32: 3.6; v=16: 2.9
32	T	3.0	50%	50%	v=16: 1.5; v=8: 1.5
64	T^{T}	19.1	100%	0%	v=64: 9.4; v=32: 6.3; v=16: 3.4
64	T	9.2	67%	33%	v=16: 5.1; v=8: 3.1; v=32: 1.0
128	T^{T}	43.9	100%	0%	v=128: 28.9; v=32: 9.5; v=64: 5.4
128	T	21.7	84%	16%	v=16: 9.4; v=32: 5.8; v=8: 3.5; v=64: 2.8; v=128: 0.3

Table 15: Digits lost at DPS when the selected phase is executed at reference precision and the rest at DPS; in parentheses the change relative to "all at DPS".

r	Stage	DPS	All at DPS	Reference chunk phase	Reference direct phase
64	T^{T}	50	17.8	3.3 (−14.5)	17.8 (0.0)
64	T	50	21.9	11.6 (−10.3)	21.9 (0.0)
128	T^{T}	90	42.6	5.2 (−37.5)	42.6 (0.0)
128	T	90	50.6	22.6 (−27.9)	51.0 (+0.4)

[10] OpenAI. Finite tensor savings and exact Fourier circuits. Preprint, 2026-09-25, OpenAI Math repository, `https://github.com/openai/math`, September 2026. Directory `preprints/Finite-tensor-savings-and-exact-Fourier-circuits-September-25-2026`.

[11] Benjamin Recht. Faster Fourier transforms. argmin (blog), 2026-10-12, `https://www.argmin.net/p/faster-fourier-transforms`, 2026. Not peer reviewed; cited as context for the critique.

[12] Lothar Reichel. Newton interpolation at Leja points. *BIT Numerical Mathematics*, 30(2): 332–346, 1990. doi: 10.1007/BF02017352.

A Additional tables

Table 16: Strict depth model at powers of two (model values, not measurements).

r	b	$d_{\mathrm{model}}(r)$	$d_{\mathrm{model}}/\log_2^4 r$
2^{12}	16	7 383 844	356.1
2^{13}	32	11 021 092	385.9
2^{14}	64	15 182 628	395.2
2^{15}	128	19 868 452	392.5
2^{16}	256	25 078 564	382.7
2^{17}	512	30 812 964	368.9

Table 17: Earlier timing measurement (`timing.py`), taken while seven other single-threaded computations ran on the machine. Columns as in Table 13.

r	Gates	Build, s	Apply, s	Mem., GiB	fft, µs	Dense, µs
64	64 800	0.16	0.01	0.04	1.1	4.0
128	928 320	0.88	0.16	0.12	1.5	14.0
256	5 051 520	4.3	0.88	0.50	2.1	57.1
512	19 638 528	15.0	3.1	1.78	3.5	222.4

Table 18: Topology audit of the strict word: 12 widths × 2 rules × 3 inputs (seed = r, another seed, h = ones). The last column states whether the structure hash is identical for the three inputs.

r	Rule	Depth	Layers	Gates	Identical hash
64	adaptive	37668	50464	64 800	yes (3 inputs)
64	chunk 1	37668	50464	64 800	yes (3 inputs)
128	adaptive	133924	197184	928 320	yes (3 inputs)
128	chunk 1	152356	215616	277 056	yes (3 inputs)
256	adaptive	518948	779392	5 051 520	yes (3 inputs)
256	chunk 1	611108	889984	1 143 936	yes (3 inputs)
288	adaptive	656793	986400	6 393 312	yes (3 inputs)
288	chunk 1	773433	1126368	1 447 776	yes (3 inputs)
320	adaptive	811515	1239360	7 920 960	yes (3 inputs)
320	chunk 1	955515	1412160	1 815 360	yes (3 inputs)
336	adaptive	626622	1098096	10 059 168	yes (3 inputs)
336	chunk 1	1053510	1556736	2 001 216	yes (3 inputs)
352	adaptive	497926	1015392	8 050 752	yes (3 inputs)
352	chunk 1	1156166	1708480	2 196 288	yes (3 inputs)
384	adaptive	592581	1208640	9 581 376	yes (3 inputs)
384	chunk 1	1375941	2033472	2 614 080	yes (3 inputs)
512	adaptive	1035044	2074880	19 638 528	yes (3 inputs)
512	chunk 1	2446116	3614976	4 647 168	yes (3 inputs)
640	adaptive	1617915	3285120	30 741 120	yes (3 inputs)
640	chunk 1	3822715	5691520	7 317 120	yes (3 inputs)
768	adaptive	1753797	3578496	40 617 600	yes (3 inputs)
768	chunk 1	5504709	8195712	10 536 576	yes (3 inputs)
1024	adaptive	1682212	4796928	64 639 488	yes (3 inputs)
1024	chunk 1	9786148	14569984	18 731 520	yes (3 inputs)

B Proofs and derivations

B.1 Nonvanishing of the split coefficients

Proof of Lemma 2. Since $\kappa = 1 + |\mu|^2$ is real and $\kappa \geq 1$, we have $t_1 = \kappa \neq 0$. By the reverse triangle inequality, $|t_2| = |\kappa - \mu| \geq |\kappa| - |\mu| = 1 + |\mu|^2 - |\mu|$. Completing the square,

$$1 + |\mu|^2 - |\mu| = \Big(|\mu| - \tfrac{1}{2}\Big)^2 + \tfrac{3}{4} \;\geq\; \tfrac{3}{4} > 0.$$

Hence $t_2 \neq 0$, and $t_1 + t_2 = \mu$ for every μ, including $\mu = 0$. The argument uses that $\mu\bar{\mu} = |\mu|^2$ is a nonnegative real number; it therefore does not transfer to $\mathbb{F}_p$, where the nonvanishing of t_1, t_2 is instead checked at each inversion (Section 4.2). □

B.2 Clausen asymptotics of $\log_{10} \kappa_2(D_H)$

Recall $\mathrm{Cl}_2(\varphi) = -\int_0^\varphi \log|2\sin(t/2)|\, dt$; its maximum on $[0, 2\pi]$ is at $\varphi = \pi/3$, $\mathrm{Cl}_2(\pi/3) \approx 1.01494$, and $\mathrm{Cl}_2(2\pi - \varphi) = -\mathrm{Cl}_2(\varphi)$.

Proposition 2. *For $u = e^{2\pi i/r}$ and $D_H = \mathrm{diag}(H_0, \ldots, H_{r-1})$,*

$$\log_{10} \kappa_2(D_H) = \frac{r}{\pi} \frac{\mathrm{Cl}_2(\pi/3)}{\ln 10} + o(r) \qquad (r \to \infty).$$

Proof. For a diagonal matrix $\kappa_2(D_H) = \max_k |H_k| / \min_k |H_k|$. Since $|1 - e^{2\pi i s/r}| = 2\sin(\pi s/r)$ for $0 < s < r$,

$$\ln|H_k| = \sum_{s=1}^{k} \ln\big(2\sin(\pi s/r)\big).$$

The summand is negative for $\pi s/r < \pi/6$ and for $\pi s/r > 5\pi/6$ and positive in between; hence $\ln|H_k|$ is minimal at $k \approx r/6$ and maximal at $k \approx 5r/6$. Approximating the sum by an integral (the integrand has only integrable logarithmic singularities at the endpoints, so the error of the Riemann sum is $o(r)$) and substituting $t = x/2$,

$$\ln|H_k| = \frac{r}{\pi} \int_0^{\pi k/r} \ln(2\sin t)\, dt + o(r) = -\frac{r}{2\pi}\, \mathrm{Cl}_2\Big(\frac{2\pi k}{r}\Big) + o(r),$$

uniformly in k. At $2\pi k/r = 5\pi/3$ the value is $-\frac{r}{2\pi}\mathrm{Cl}_2(5\pi/3) = \frac{r}{2\pi}\mathrm{Cl}_2(\pi/3)$, and at $2\pi k/r = \pi/3$ it is $-\frac{r}{2\pi}\mathrm{Cl}_2(\pi/3)$. The difference is $\frac{r}{\pi}\mathrm{Cl}_2(\pi/3) + o(r)$; dividing by $\ln 10$ gives the claim, with $c = \mathrm{Cl}_2(\pi/3)/(\pi \ln 10) = 0.14030576$. □

The finite values in Table 6 are exact differences of partial sums; they differ from cr by $o(r)$, e.g. by 10^{-4} at $r = 1024$.

B.3 Localisation for $\mathbb{F}_p$

Proof of Proposition 1. Let $p = kN + 1$ be prime and $\omega \in \mathbb{F}_p$ of multiplicative order N. Since $p \nmid N$, ω is a root of Φ_N modulo p, and $\zeta_N \mapsto \omega$ defines a surjective ring homomorphism $\phi : \mathbb{Z}[\zeta_N] \cong \mathbb{Z}[x]/(\Phi_N) \to \mathbb{F}_p$ with kernel $\mathfrak{p}$, a prime ideal. Let $R = \mathbb{Z}[\zeta_N]_{\mathfrak{p}} \subset \mathbb{Q}(\zeta_N)$ be the localisation, i.e. the fractions a/b with $a, b \in \mathbb{Z}[\zeta_N]$ and $b \notin \mathfrak{p}$. Then ϕ extends uniquely to a ring homomorphism $\phi : R \to \mathbb{F}_p$, $\phi(a/b) = \phi(a)\phi(b)^{-1}$; rational constants with denominators prime to p lie in R.

We show by induction along the straight-line program that every intermediate value z lies in R and that the value computed in $\mathbb{F}_p$ is $\phi(z)$. Inputs and constants satisfy this. Ring operations preserve R and commute with ϕ. If $z \in R$ is inverted and the $\mathbb{F}_p$ computation inverts $\phi(z) \neq 0$, then $z \notin \mathfrak{p}R$, so z is a unit of the local ring R, $z^{-1} \in R$ and $\phi(z^{-1}) = \phi(z)^{-1}$. Thus the $\mathbb{F}_p$

Table 19: Lean definitions versus the preprint. G = `Goal.lean`, R = `RAM.lean`, C = `FourierCircuit/Core.lean`.

Lean definition (file)	Content	Verdict
`zeta`, `fourierMatrix` (C); `dft` (G)	$\zeta_n = \exp(2\pi i/n)$; (ζ_n^{jk}); unnormalised $F_n x$	Matches `introduction.tex`
Constants m, W_*, Δ, λ, θ (G)	Same exact constants as (1)	Matches
`paperTime`, `decimalTime` (G)	$n(\ln n)^{\theta}(\ln\ln n)^{4-\theta}$; exponent $1-10^{-13}$; natural logarithms	Matches `thm:main`, `cor:decimal`
`DFTProgram` (G)	For all $n>0$: root order $0<D<1024\,n^3$ computed and charged in the model; `d.work` $+ 1 +$ `result.work` $\le W(n)$; result = `dft` $n\,x$ for all x	Matches; root order is computed in the model (stronger)
`TimeBounds`, `DFTGoal` (G)	$W = O($`paperTime`$)$, $W = O($`decimalTime`$)$, $W = o(n\log n)$; fixed `order`, `solve`, W	Matches
`ConvGoal` (G)	Bilinear products allowed (`Prog true`); result is the linear convolution	Matches; stronger (root bound, bilinear products only)
`Paint`, `Bill`, atoms (R)	Separate scalar and data registers; every atom costs 1; `inv` only on scalars with nonzero condition; no complex constants or $\mathbb{C}\leftrightarrow\mathbb{N}$ atoms; branching on integers only	Matches; no simulation theorem for word-RAM
`receipt`	$D =$ `receipt`$(n) < 8\,n^3$ in the `_order` variants	Formula differs from D_* (`eq:master-root`); bound satisfied
Internal network (`lunch_big`, `alpha_theta`)	32 binary coordinates per factor, 4060 triples, bound 2^{50}; $W = O\big(n(\ln n)^{\alpha}(\ln\ln n)^2\big)$, $\alpha = 1-2\cdot10^{-11} < \theta$	Different from `network.tex`; Δ and `net:tensor-bound` not checked
Axioms (all four main theorems)	`propext`, `Classical.choice`, `Quot.sound`	No `sorry`; 89 modules in the import closure

output is the image of the $\mathbb{Q}(\zeta_N)$ output. If the latter equals $F_r(\zeta_r)$, the former equals $F_r(\phi(\zeta_r))$; contrapositively, a mismatch in $\mathbb{F}_p$ implies a mismatch over $\mathbb{Q}(\zeta_N)$, whereas agreement in $\mathbb{F}_p$ does not exclude a nonzero difference lying in $\mathfrak{p}R$.

Complex conjugation on $\mathbb{Q}(\zeta_N)$ is the automorphism $\sigma : \zeta_N \mapsto \zeta_N^{-1}$. The DAG evaluated with inverse roots computes $\sigma(\mu) = \bar{\mu}$ over $\mathbb{Q}(\zeta_N)$, and the same DAG evaluated in $\mathbb{F}_p$ with ω^{-1} in place of ω computes $\phi(\sigma(\mu))$ by the argument above. Hence the $\mathbb{F}_p$ check of the C-slot path tests the image of the same conjugate expression, although $\mathbb{F}_p$ itself has no nontrivial automorphism. □

C Summary of the Lean statement audit

Table 19 summarises the comparison of the Lean definitions with the preprint (Section 4.8). Proof bodies were not reread; their correctness for these definitions is checked by the Lean kernel.